

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ KỸ THUẬT TP. HỒ CHÍ MINH
KHOA ĐIỆN ĐIỆN TỬ
BỘ MÔN KỸ THUẬT MÁY TÍNH - VIỄN THÔNG

ĐỒ ÁN MÔN HỌC 2

**THIẾT KẾ VÀ KIỂM THỬ
BỘ CHUYỂN ĐỔI AXI4 SANG APB4
SỬ DỤNG UVM**

NGÀNH CÔNG NGHỆ KỸ THUẬT ĐIỆN TỬ - TRUYỀN THÔNG

Sinh viên: **NGUYỄN HOÀNG DIỄM CHI**

MSSV: 22161004

NGUYỄN ĐÌNH KHÁNH VY

MSSV: 22161043

TP. HỒ CHÍ MINH – 01/2026

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ KỸ THUẬT TP. HỒ CHÍ MINH
KHOA ĐIỆN ĐIỆN TỬ
BỘ MÔN KỸ THUẬT MÁY TÍNH - VIỄN THÔNG

ĐỒ ÁN MÔN HỌC 2

**THIẾT KẾ VÀ KIỂM THỬ BỘ CHUYỂN ĐỔI
AXI4 SANG APB4 SỬ DỤNG UVM**

NGÀNH CÔNG NGHỆ KỸ THUẬT ĐIỆN TỬ TRUYỀN THÔNG

Sinh viên: **NGUYỄN HOÀNG DIỄM CHI**
MSSV: 22161004
NGUYỄN ĐÌNH KHÁNH VY
MSSV: 22161043

Hướng dẫn: **ThS. NGUYỄN VĂN PHÚC**

TP. HỒ CHÍ MINH – 01/2026

LỜI CẢM ƠN

Nhóm em xin bày tỏ lòng biết ơn sâu sắc đến Thầy Nguyễn Văn Phúc - người đã trực tiếp hướng dẫn đồ án này. Với sự nhiệt huyết và tinh thần trách nhiệm, thầy đã tận tâm hỗ trợ, nhiệt tình hướng dẫn nhóm chúng em trong suốt thời gian qua, giúp chúng em định hướng được đề tài, vạch ra kế hoạch và thầy đã chỉ ra một số vấn đề mà nhóm chúng em mắc phải. Sự đồng hành và định hướng từ Thầy là nguồn động lực to lớn giúp chúng em có động lực rất lớn, vượt qua những khó khăn trong suốt quá trình thực hiện đồ án.

Nhóm em cũng xin chân thành cảm ơn Ban Giám hiệu nhà trường và Bộ môn Kỹ thuật Máy tính - Viễn thông đã tạo điều kiện thuận lợi về cơ sở vật chất, môi trường học tập chuyên nghiệp và hỗ trợ sinh viên trong việc triển khai và thực hiện đồ án môn học.

Chúng em xin chân thành cảm ơn!

Trân trọng

Nhóm sinh viên thực hiện

Nguyễn Hoàng Diễm Chi

Nguyễn Đình Khánh Vy

TÓM TẮT

Trong các hệ thống vi mạch hiện đại, đặc biệt là hệ thống trên chip (SoC), việc truyền nhận dữ liệu giữa các khối chức năng như bộ xử lý trung tâm, bộ nhớ và các ngoại vi đóng vai trò then chốt trong việc đảm bảo hiệu suất tổng thể và độ tin cậy của hệ thống. Để thực hiện điều này, các giao thức truyền thông nội bộ được sử dụng nhằm kết nối và điều phối hoạt động giữa các thành phần cứng với nhau một cách hiệu quả.

Trong các giao thức truyền thông nội bộ hiện nay, chuẩn giao thức hệ AMBA (Advanced Microcontroller Bus Architecture) do hãng ARM phát triển đã trở thành chuẩn phổ biến trong các thiết kế SoC. Trong đó, AXI4 (Advanced eXtensible Interface) được sử dụng để kết nối các khối yêu cầu băng thông cao như CPU hoặc DMA, trong khi APB (Advanced Peripheral Bus) được dùng cho các ngoại vi có tốc độ thấp. Tuy nhiên, do sự khác biệt về đặc tính và cơ chế truyền dữ liệu, việc thiết kế bộ chuyển đổi giữa AXI4 sang APB4 là cần thiết. Và để đảm bảo bộ chuyển đổi hoạt động đúng theo đặc tả hãng AMBA trong mọi kịch bản truyền và nhận dữ liệu, việc xây dựng môi trường kiểm thử là một yếu tố cần thiết. Với phương pháp xác minh tiêu chuẩn UVM (Universal Verification Methodology) dùng để mô phỏng, kiểm thử và đánh giá toàn diện chức năng của thiết kế, có khả năng mở rộng cao là lựa chọn phù hợp để kiểm chứng thiết kế.

Đề tài “Thiết kế và kiểm thử bộ chuyển đổi AXI4 sang APB4 sử dụng UVM” được thực hiện với mục tiêu thiết kế bộ chuyển đổi giao thức AXI4 sang APB4 sử dụng ngôn ngữ Verilog, tuân thủ đặc tả chuẩn AMBA, đồng thời xây dựng môi trường xác minh tự động dựa trên phương pháp UVM sử dụng phần MobaXterm kết hợp với QuestaSim. Trong môi trường này, AXI VIP được sử dụng đóng vai trò Master và APB VIP đóng vai trò Slave để mô phỏng, kiểm thử và đánh giá hoạt động của bộ chuyển đổi thông qua các sóng tín hiệu, đánh giá các thông số độ bao phủ và độ bao phủ chức năng. Ngoài ra, đề tài còn đánh giá các thông số kỹ thuật của bộ chuyển đổi sau tổng hợp sử dụng phần mềm XILINX VIVADO. Tuy nhiên, phạm vi của đề tài mới dừng lại ở mức thiết kế và mô phỏng phần mềm, chưa được triển khai kiểm chứng thực tế trên phần cứng.

MỤC LỤC

DANH MỤC HÌNH	VIII
DANH MỤC BẢNG	X
CÁC TỪ VIẾT TẮT	XI
CHƯƠNG 1 TỔNG QUAN.....	1
1.1 GIỚI THIỆU	1
1.2 MỤC TIÊU NGHIÊN CỨU	2
1.3 TÌNH HÌNH NGHIÊN CỨU	3
1.4 BỐ CỤC CỦA ĐỀ TÀI.....	4
CHƯƠNG 2 CƠ SỞ LÝ THUYẾT	5
2.1 TỔNG QUAN VỀ HỆ THỐNG TRÊN CHIP	5
2.2 TỔNG QUAN VỀ HỆ THỐNG BUS TRONG SOC	6
2.2.1 Tổng quan về giao thức AMBA AXI.....	6
2.2.1.1 Giới thiệu chung về giao thức AMBA AXI.....	6
2.2.1.2 Cấu trúc bus AXI.....	7
2.2.1.3 Giới thiệu về giao thức AMBA AXI4	11
2.2.1.4 Các quy định về hoạt động	12
2.2.1.5 Cơ chế bắt tay của AXI4	13
2.2.1.6 Cơ chế burst của AXI4.....	15
2.2.2 Tổng quan về giao thức AMBA APB	17
2.2.2.1 Giới thiệu chung về giao thức AMBA APB.....	17
2.2.2.2 Tín hiệu của giao thức APB.....	17
2.2.2.3 Các quy định về hoạt động	18
2.3 TỔNG QUAN VỀ PHƯƠNG THỨC XÁC MINH UVM	19
2.3.1 Cấu trúc UVM Testbench.....	20
2.3.2 Lớp cấu trúc UVM Testbench	21
2.3.2.1 UVM Testbench Top	21
2.3.2.2 UVM Test.....	21
2.3.2.3 UVM Environment	22
2.3.2.4 UVM Agent	22

2.3.2.5	<i>UVM Sequencer</i>	22
2.3.2.6	<i>UVM Sequence</i>	22
2.3.2.7	<i>UVM Driver</i>	23
2.3.2.8	<i>UVM Monitor</i>	23
2.3.2.9	<i>UVM Scoreboard</i>	23
2.4	CƠ CHẾ UVM	23
2.4.1	Cơ chế Pha	23
2.4.2	Cơ chế Factory	24
2.4.3	Cơ chế Config_db	25
2.4.4	Chức năng độ bao phủ và độ bao phủ chức năng	25
CHƯƠNG 3	THIẾT KẾ HỆ THỐNG	26
3.1	YÊU CẦU HỆ THỐNG	26
3.2	THIẾT KẾ BỘ CHUYỂN ĐỔI AXI4 SANG APB4	26
3.2.1	Khởi Asynchronous FIFO.....	31
3.2.1.1	<i>Phân tích chi tiết khối AW_FIFO</i>	33
3.2.1.2	<i>Phân tích chi tiết khối WR_FIFO</i>	33
3.2.1.3	<i>Phân tích chi tiết khối AR_FIFO</i>	34
3.2.1.4	<i>Phân tích chi tiết khối RD_FIFO</i>	35
3.2.2	Thiết kế khối APB master.....	35
3.2.2.1	<i>Thiết kế khối RW Arbiter</i>	39
3.2.2.2	<i>Thiết kế khối giải mã địa chỉ</i>	41
3.3	KIẾN TRÚC MÔI TRƯỜNG KIỂM THỬ UVM	43
CHƯƠNG 4	KẾT QUẢ THỰC HIỆN	46
4.1	TÓM TẮT NỘI DUNG	46
4.2	MÔ PHỎNG HỆ THỐNG	46
4.2.1	Hoạt động ghi dữ liệu	48
4.2.1.1	<i>Tập kiểm tra 1</i>	49
4.2.1.2	<i>Tập kiểm tra 2</i>	50
4.2.1.3	<i>Tập kiểm tra 3</i>	51
4.2.1.4	<i>Tập kiểm tra 4</i>	52

4.2.2	Hoạt động đọc dữ liệu	53
4.2.2.1	<i>Tập kiểm tra 5</i>	54
4.2.2.2	<i>Tập kiểm tra 6</i>	55
4.2.2.3	<i>Tập kiểm tra 7</i>	56
4.2.2.4	<i>Tập kiểm tra 8</i>	57
4.2.3	Hoạt động đọc và ghi dữ liệu (Testcase 9)	58
4.3	KẾT QUẢ BÁO CÁO ĐỘ BAO PHỦ VÀ ĐỘ BAO PHỦ CHỨC	
	NĂNG	60
4.4	ĐÁNH GIÁ TÀI NGUYÊN SỬ DỤNG	61
4.5	ĐÁNH GIÁ TẦN SỐ VÀ CÔNG SUẤT CỦA THIẾT KẾ.....	62
CHƯƠNG 5	KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	64
5.1	KẾT LUẬN	64
5.2	HƯỚNG PHÁT TRIỂN	65
	TÀI LIỆU THAM KHẢO.....	66
	PHỤ LỤC A	68
	PHỤ LỤC B	70

DANH MỤC HÌNH

Hình 2.1. Giao diện giao tiếp và kết nối bus.....	7
Hình 2.2. Các kênh giao tiếp giữa Master và Slave	8
Hình 2.3. Hoạt động của giao dịch đọc	12
Hình 2.4. Hoạt động của giao dịch ghi	12
Hình 2.5. Tín hiệu VALID tích cực trước tín hiệu READY	14
Hình 2.6. Tín hiệu VALID tích cực sau tín hiệu READY	14
Hình 2.7. Tín hiệu VALID tích cực cùng lúc với tín hiệu READY	14
Hình 2.8. Minh họa cơ chế burst	15
Hình 2.9. Máy trạng thái mô tả hoạt động giao thức APB	19
Hình 2.10. Cấu trúc điển hình của UVM.....	20
Hình 2.11. Phân cấp UVM.....	21
Hình 2.12. UVM Agent	22
Hình 2.13. Cơ chế pha trong UVM	24
Hình 3.1. Sơ đồ khối của cầu chuyển đổi	27
Hình 3.2. Sơ đồ tín hiệu kết nối của khối chính.....	28
Hình 3.3. Sơ đồ tín hiệu kết nối của khối Asynchronous FIFO.....	31
Hình 3.4. Sơ đồ nguyên lý khối AW_FIFO	33
Hình 3.5. Sơ đồ nguyên lý khối WR_FIFO	34
Hình 3.6. Sơ đồ nguyên lý khối AR_FIFO	34
Hình 3.7. Sơ đồ nguyên lý khối RD_FIFO	35
Hình 3.8. Sơ đồ tín hiệu kết nối của khối APB master.....	36
Hình 3.9. Sơ đồ tín hiệu kết nối của khối RW Arbiter	40
Hình 3.10. Máy trạng thái của khối RW Arbiter.....	41
Hình 3.11. Sơ đồ tín hiệu kết nối của khối giải mã địa chỉ.....	42
Hình 3.12. Sơ đồ kiến trúc của khối giải mã địa chỉ	43
Hình 3.13. Kiến trúc môi trường kiểm thử UVM cho bộ chuyển đổi.....	44
Hình 4.1. Dạng sóng tệp kiểm tra 1 của quá trình ghi.....	49
Hình 4.2. Dạng sóng tệp kiểm tra 2 của quá trình ghi.....	51
Hình 4.3. Dạng sóng tệp kiểm tra 3 của quá trình ghi.....	52

Hình 4.4. Dạng sóng tệp kiểm tra 4 của quá trình ghi.....	53
Hình 4.5. Dạng sóng tệp kiểm tra 5 của quá trình đọc	54
Hình 4.6. Dạng sóng tệp kiểm tra 6 của quá trình đọc	55
Hình 4.7. Dạng sóng tệp kiểm tra 7 của quá trình đọc	56
Hình 4.8. Dạng sóng tệp kiểm tra 8 của quá trình đọc	57
Hình 4.9. Dạng sóng tệp kiểm tra 9 của quá trình đọc và ghi.....	58
Hình 4.10. Dạng sóng tệp kiểm tra 9 của quá trình đọc	59
Hình 4.11. Dạng sóng tệp kiểm tra 9 của giao dịch ghi AXI4.....	59
Hình 4.12. Dạng sóng tệp kiểm tra 9 của giao dịch ghi APB4.....	60
Hình 4.13. Kết quả báo cáo độ bao phủ của bộ chuyển đổi	60
Hình 4.14. Kết quả báo cáo độ bao phủ chức năng của bộ chuyển đổi.....	61
Hình phụ lục 1. Minh chứng tài nguyên sử dụng của thiết kế	68
Hình phụ lục 2. Minh chứng công suất tiêu thụ ở tần số 200MHz, 100MHz.....	68
Hình phụ lục 3. Minh chứng định thời ở tần số 200MHz, 100MHz.....	68
Hình phụ lục 4. Minh chứng công suất tiêu thụ ở tần số 125MHz, 50MHz.....	69
Hình phụ lục 5. Minh chứng công suất tiêu thụ ở tần số 25MHz, 20MHz	69

DANH MỤC BẢNG

Bảng 2.1. Các tín hiệu của kênh địa chỉ ghi.....	9
Bảng 2.2. Các tín hiệu của kênh dữ liệu ghi	9
Bảng 2.3. Các tín hiệu của kênh phản hồi ghi.....	10
Bảng 2.4. Các tín hiệu của kênh địa chỉ đọc	10
Bảng 2.5. Các tín hiệu của kênh dữ liệu đọc.....	11
Bảng 2.6. Các cặp tín hiệu bắt tay trong giao thức AXI4.....	13
Bảng 2.7. Các loại burst tương ứng với giá trị AxBURST	16
Bảng 2.8. Kích thước dữ liệu tương ứng với giá trị AxLEN	17
Bảng 2.9. Tín hiệu của giao thức APB	18
Bảng 3.1. Các tín hiệu kết nối của bộ chuyển đổi AXI4 sang APB4.....	29
Bảng 3.2. Chức năng của các tín hiệu trong khối Asynchronous FIFO	32
Bảng 3.3. Chức năng của các tín hiệu trong khối APB Master	36
Bảng 3.4. Mô tả điều kiện vùng địa chỉ ánh xạ tới slave tương ứng.....	39
Bảng 3.5. Chức năng của các tín hiệu trong khối giải mã địa chỉ.....	42
Bảng 4.1. Giới thiệu các testcase.....	47
Bảng 4.2. Thông số đầu vào của tệp kiểm tra 1	49
Bảng 4.3. Thông số đầu vào của tệp kiểm tra 2	50
Bảng 4.3. Thông số đầu vào của tệp kiểm tra 3	51
Bảng 4.4. Thông số đầu vào của tệp kiểm tra 4	52
Bảng 4.6. Thông số đầu vào của tệp kiểm tra 5	54
Bảng 4.7. Thông số đầu vào của tệp kiểm tra 6	55
Bảng 4.8. Thông số đầu vào của tệp kiểm tra 7	56
Bảng 4.8. Thông số đầu vào của tệp kiểm tra 8	57
Bảng 4.9. Thông số đầu vào của tệp kiểm tra 9	58
Bảng 4.10. Thông số tài nguyên hệ thống.....	61
Bảng 4.11. Công suất của thiết kế với các tần số khác nhau	62
Bảng 4.12. Báo cáo về mặt timing của hệ thống.....	62

CÁC TỪ VIẾT TẮT

AMBA	Advanced Microcontroller Bus Architecture
APB	Advanced Peripheral Bus
ARM	Advanced RISC Machine
AXI	Advanced eXtensible Interface
CPU	Central Processing Unit
DMA	Direct Memory Access
FIFO	First In First Out
SoC	System-on-Chip
UVM	Universal Verification Methodology
VIP	Verification Intellectual Property
DUT	Device Under Test
TLM	Transaction-Level Modeling
RTL	Register Transfer Level
FSM	Finite State Machine
SVA	SystemVerilog Assertions
OVM	Open Verification Methodology
VMM	Verification Methodology Manual
eRM	e Reuse Methodology
AVM	Advanced Verification Methodology
EDA	Electronic Design Automation
IEEE	Institute of Electrical and Electronics Engineers
FPGA	Field Programmable Gate Array

CHƯƠNG 1

TỔNG QUAN

1.1 GIỚI THIỆU

Trong kỷ nguyên của các hệ thống trên chip hiện đại, việc tích hợp hàng chục, thậm chí hàng trăm khối IP chức năng khác nhau như bộ xử lý, bộ nhớ và các ngoại vi trên cùng một chip đã trở thành xu hướng nhằm nâng cao hiệu năng và giảm chi phí hệ thống. Để các khối IP này có thể giao tiếp và phối hợp hoạt động hiệu quả, các chuẩn bus nội bộ đóng vai trò then chốt, trong đó, chuẩn giao tiếp họ AMBA do hãng ARM phát triển đã trở thành tiêu chuẩn công nghiệp được sử dụng rộng rãi trong thiết kế SoC.

Đối với kiến trúc SoC hiện nay, các khối chức năng có đặc tính tốc độ và yêu cầu băng thông rất khác nhau. Giao thức AXI4 được thiết kế cho các thành phần yêu cầu băng thông cao và độ trễ thấp như CPU, DMA và nhiều khối khác, với khả năng hỗ trợ truyền dữ liệu và cơ chế pipeline. Ngược lại, giao thức APB4 có cấu trúc đơn giản, tiêu thụ năng lượng thấp và phù hợp với các ngoại vi tốc độ thấp như UART, GPIO hay Timer. Sự khác biệt này đặt ra yêu cầu cần có một cơ chế trung gian để kết nối các ngoại vi cấu hình theo APB4 vào hệ thống bus AXI4 tốc độ cao.

Do đó, việc thiết kế một bộ chuyển đổi giao thức từ AXI4 sang APB4 là cần thiết nhằm đảm bảo khả năng giao tiếp, đồng bộ và mở rộng hệ thống trên chip. Tuy nhiên, thách thức kỹ thuật không chỉ nằm ở việc thiết kế phần cứng mà còn ở công đoạn kiểm thử và xác minh chức năng. Với độ phức tạp cao của giao thức AXI4, các phương pháp kiểm thử truyền thống khó có thể bao quát đầy đủ các kịch bản truyền dữ liệu và các trường hợp ẩn, khó phát hiện, dẫn đến nguy cơ tiềm ẩn lỗi trong thiết kế.

Trước tình hình đó, phương pháp xác minh tiêu chuẩn UVM được xem là giải pháp tối ưu, cho phép xây dựng môi trường kiểm thử tự động, có khả năng tái sử dụng và mở rộng cao. Đồng thời, phương pháp có thể đánh giá thiết kế một cách hệ thống và toàn diện, bao phủ toàn bộ thiết kế. Việc áp dụng UVM không chỉ giúp nâng cao độ tin cậy của thiết kế mà còn tiệm cận với quy trình xác minh trong công nghiệp vi mạch hiện nay.

Xuất phát từ những yêu cầu và thách thức nêu trên, đề tài “Thiết kế và kiểm thử bộ chuyển đổi AXI4 sang APB4 sử dụng UVM” được thực hiện với mục tiêu thiết kế bộ chuyển đổi giao thức tuân thủ đặc tả chuẩn AMBA sử dụng ngôn ngữ Verilog, đồng thời xây dựng môi trường xác minh dựa trên phương pháp UVM. Trong quá trình thực hiện, đề tài tập trung vào việc xây dựng kiến trúc bộ chuyển đổi, thiết lập môi trường kiểm thử với AXI VIP đóng vai trò Master và APB VIP đóng vai trò Slave, cũng như mô phỏng và đánh giá hoạt động của thiết kế thông qua phân tích dạng sóng và độ phủ chức năng. Sau đó, tiến hành phân tích tài nguyên cũng như các thông số kỹ thuật của thiết kế sau tổng hợp. Phạm vi nghiên cứu của đề tài hiện tại giới hạn ở mức thiết kế phần cứng và mô phỏng, chưa triển khai kiểm chứng trên phần cứng thực tế.

1.2 MỤC TIÊU NGHIÊN CỨU

Mục tiêu của đề tài này là thiết kế bộ chuyển đổi giao thức AXI4 sang APB4 tuân thủ đặc tả AMBA của hãng ARM, đồng thời xây dựng môi trường xác minh tự động dựa trên phương pháp xác minh UVM nhằm đảm bảo tính đúng đắn và hiệu quả của bộ chuyển đổi. Các mục tiêu chính của đề tài như sau:

- Thiết kế và xây dựng bộ chuyển đổi AXI4 sang APB4 sử dụng ngôn ngữ Verilog, đảm bảo tuân thủ đặc tả kỹ thuật do hãng ARM đề ra.
- Thiết kế môi trường xác minh UVM có khả năng kiểm tra, xác minh được toàn bộ các chức năng của bộ chuyển đổi.
- Đưa ra các tệp kiểm tra toàn bộ các chức năng cơ bản của bộ chuyển đổi AXI4 sang APB4, kết quả thể hiện trên sóng tín hiệu.
- Đánh giá độ bao phủ và độ bao phủ chức năng khi kiểm thử thiết kế.
- Trình bày kết quả và đánh giá các thông số kỹ thuật của bộ chuyển đổi.

1.3 TÌNH HÌNH NGHIÊN CỨU

Trong bối cảnh các hệ thống trên chip ngày càng phát triển theo hướng tích hợp cao, việc sử dụng các giao thức bus nội bộ theo chuẩn họ AMBA đã trở thành giải pháp phổ biến nhằm đáp ứng đồng thời yêu cầu về hiệu năng và tiêu thụ năng lượng. Tuy nhiên, sự khác biệt về đặc tính truyền dữ liệu giữa các bus tốc độ cao và các bus ngoại vi tốc độ thấp đặt ra nhu cầu thiết kế các bộ chuyển đổi giao thức trung gian, đồng thời làm tăng đáng kể độ phức tạp trong công tác kiểm thử và xác minh thiết kế. Đặc biệt, với giao thức AXI4 có kiến trúc đa kênh và cơ chế truyền dữ liệu phức tạp, các phương pháp kiểm thử truyền thống khó có thể bao quát đầy đủ kịch bản hoạt động và các trường hợp ẩn khó phát hiện.

Trước thực tế đó, nhiều nghiên cứu trong và ngoài nước đã lựa chọn áp dụng UVM là phương pháp xác minh tiêu chuẩn cho phép xây dựng môi trường kiểm thử tự động, hỗ trợ phát sinh stimulus ngẫu nhiên có ràng buộc và đánh giá thiết kế dựa trên các thông số độ bao phủ, qua đó giúp phát hiện lỗi một cách hệ thống và hiệu quả hơn.

Cụ thể trong bài báo [1], nhóm tác giả đã tiến hành so sánh giữa phương pháp kiểm thử truyền thống sử dụng SystemVerilog và phương pháp dựa trên UVM trong việc xác minh cơ chế phân xử của giao thức AXI4-Lite. Kết quả cho thấy môi trường UVM đạt được độ bao phủ cao hơn và có khả năng xử lý tốt các kịch bản phức tạp liên quan đến tranh chấp bus, từ đó khẳng định của UVM khi áp dụng cho các thiết kế giao thức AMBA có độ phức tạp cao.

Trong bài báo [2], tác giả tập trung vào việc xây dựng môi trường UVM để kiểm thử giao thức AMBA APB. Tác giả nhấn mạnh vai trò của constrained-random stimulus và độ bao phủ chức năng trong việc đảm bảo giao thức APB hoạt động đúng trong nhiều kịch bản khác nhau.

Trong một hướng nghiên cứu khác, luận văn [3] tập trung vào thiết kế phần cứng cho cầu nối APB sang AHB và xây dựng môi trường xác minh dựa trên UVM. Trong đó, trình bày chi tiết cách tổ chức các thành phần có trong testbench như driver, monitor, scoreboard và cơ chế thu thập độ bao phủ chức năng nhằm đảm bảo thiết kế tuân thủ đặc tả AMBA.

Trong bài báo [4], các tác giả tập trung vào việc xây dựng môi trường môi trường xác minh UVM cho bộ chuyển đổi AHB sang APB. Trong đó, tác giả đề xuất kiến trúc testbench sử dụng interface được mã hóa kết hợp với mô hình reactive slave. Mặc dù đối tượng nghiên cứu không phải là bộ chuyển đổi AXI4 sang APB4 nhưng nghiên cứu này cho thấy khả năng mở rộng và tái sử dụng môi trường UVM khi áp dụng cho các bộ chuyển đổi giao thức bus trong SoC.

1.4 BỐ CỤC CỦA ĐỀ TÀI

Chương 1. Giới thiệu: Trình bày tổng quan về đề tài, mục tiêu đề tài, tình hình nghiên cứu và phương pháp nghiên cứu được sử dụng trong đồ án môn học.

Chương 2. Cơ sở lý thuyết: Tổng quan về bus của SoC và AMBA, và lý thuyết cơ bản và vai trò của giao thức AXI4, APB4 và phương pháp xác minh thiết kế UVM.

Chương 3. Thiết kế hệ thống: Trình bày chi tiết về sơ đồ khối của bộ chuyển đổi, nguyên lý hoạt động của các khối thành phần bên trong. Thiết kế môi trường UVM và kiểm thử bộ chuyển đổi sử dụng UVM.

Chương 4. Kết quả thực hiện: Trình bày kết quả sau khi tổng hợp thiết kế, mô phỏng chức năng, kiểm tra và đánh giá các thông số liên quan đến thiết kế.

Chương 5. Kết luận và hướng phát triển: Kết luận, tổng kết về đề tài với những kết quả đạt được, những mặt hạn chế và đề xuất hướng phát triển đề tài.

CHƯƠNG 2

CƠ SỞ LÝ THUYẾT

2.1 TỔNG QUAN VỀ HỆ THỐNG TRÊN CHIP

Hệ thống trên chip là một vi mạch tích hợp gần như toàn bộ các khối chức năng cần thiết để tạo thành một hệ thống điện tử hoàn chỉnh. Một SoC điển hình bao gồm bộ xử lý trung tâm, hệ thống bộ nhớ, các cổng vào, cổng ra, bộ điều khiển giao tiếp, các khối xử lý tín hiệu tương tự, và các khối chức năng chuyên biệt khác. Hệ thống trên chip khác với kiến trúc máy tính truyền thống, nó hợp nhất toàn bộ các thành phần này vào một vi mạch duy nhất. Cách tiếp cận này giúp giảm kích thước hệ thống, tiết kiệm điện năng, tăng hiệu năng xử lý và giảm độ trễ truyền dữ liệu giữa các khối chức năng [5].

Tùy thuộc vào mục đích sử dụng và yêu cầu kỹ thuật của từng ứng dụng, SoC có thể tích hợp thêm các chức năng chuyên biệt. Trong các thiết bị di động, hệ thống IoT hoặc hệ thống nhúng thông minh, SoC thường tích hợp thêm các khối truyền thông không dây như WiFi, Bluetooth hoặc các khối thu phát vô tuyến. Việc tích hợp trực tiếp các thành phần này trên cùng một chip giúp rút ngắn khoảng cách kết nối nội bộ, từ đó làm giảm tiêu thụ năng lượng, cải thiện tốc độ truyền dữ liệu và nâng cao độ tin cậy của toàn bộ hệ thống.

Nhờ mức độ tích hợp cao, SoC mang lại nhiều ưu điểm nổi bật như kích thước nhỏ gọn, thiết kế phần cứng đơn giản, giảm số lượng linh kiện rời và tiết kiệm năng lượng. Điều này giúp rút ngắn thời gian phát triển sản phẩm, giảm chi phí sản xuất và nâng cao hiệu quả kinh tế, đặc biệt khi sản xuất với số lượng lớn. Ngoài ra, SoC có số lượng kết nối vật lý ít hơn so với các hệ thống đa linh kiện truyền thống, từ đó làm giảm khả năng xảy ra lỗi kết nối, tăng độ ổn định và độ tin cậy trong quá trình vận hành [6], [7].

Tuy nhiên, việc tích hợp như vậy cũng có một số hạn chế nhất định. Mỗi SoC thường được thiết kế để tối ưu cho một nhóm ứng dụng hoặc phân khúc sản phẩm cụ thể, do đó tính linh hoạt không cao. Nếu tích hợp quá nhiều chức năng, chi phí chip và giá thành sản phẩm sẽ tăng trong khi một số chức năng không được sử dụng. Ngược lại, nếu tích hợp quá ít chức năng, SoC có thể không đáp ứng được yêu cầu ứng dụng hoặc làm tăng độ phức tạp trong thiết kế hệ thống, kéo dài thời gian đưa sản phẩm ra thị trường [6].

2.2 TỔNG QUAN VỀ HỆ THỐNG BUS TRONG SOC

Trong hệ thống trên chip, bus nội bộ đóng vai trò xương sống kết nối giữa các khối chức năng như bộ xử lý, bộ nhớ và các thiết bị ngoại vi. Hệ thống bus quyết định trực tiếp đến hiệu năng truyền dữ liệu, độ trễ, khả năng mở rộng và mức độ tiêu thụ năng lượng của toàn bộ hệ thống chip. Do đặc điểm các khối chức năng trong SoC có yêu cầu khác nhau về tốc độ và băng thông, kiến trúc bus thường được tổ chức theo dạng phân cấp, trong đó các bus tốc độ cao phục vụ khối xử lý và bộ nhớ, còn các bus đơn giản hơn được dùng cho ngoại vi. Chuẩn giao thức họ AMBA do hãng ARM đề xuất là một trong những kiến trúc bus phổ biến nhất hiện nay. Trong đề án này, chúng tôi sẽ tập trung vào hai giao thức tiêu biểu là AXI và APB.

2.2.1 Tổng quan về giao thức AMBA AXI

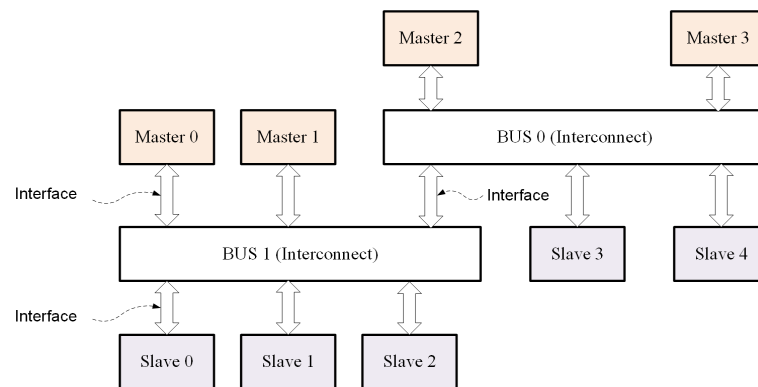
2.2.1.1 Giới thiệu chung về giao thức AMBA AXI

Giao thức AXI là một giao thức bus thuộc chuẩn họ AMBA do hãng ARM phát triển, được sử dụng rộng rãi trong các hệ thống trên chip. Phiên bản đầu tiên của AXI là AXI3, ra đời vào năm 2003, đánh dấu bước tiến quan trọng trong việc cải thiện hiệu năng giao tiếp nội bộ trong SoC. Trong giai đoạn 2010-2011, hãng ARM tiếp tục phát triển AXI thành các biến thể AXI4 Lite và AXI4 Stream. AXI4 Lite là phiên bản đơn giản hóa, phù hợp cho các giao dịch cấu hình và điều khiển có độ trễ thấp, trong khi AXI4 Stream được thiết kế cho việc truyền dữ liệu liên tục không cần địa chỉ, thường dùng trong các ứng dụng xử lý tín hiệu số, hình ảnh và video. Giai đoạn 2013-2019, phiên bản AXI5 ra đời với nhiều cải

tiến về hiệu năng, bảo mật và khả năng đồng bộ, đáp ứng yêu cầu ngày càng cao của các hệ thống SoC hiện đại.

Giao thức AXI là giao thức mở, hỗ trợ nhiều cơ chế truyền dữ liệu và được tối ưu cho các hệ thống có yêu cầu cao về tần số hoạt động, băng thông và độ trễ thấp. Do cùng thuộc chuẩn họ AMBA, AXI tương thích tốt với các giao thức bus khác như AHB hay APB, đơn giản hóa quá trình tích hợp hệ thống [8].

Giao thức AXI quy định chuẩn giao tiếp giữa Master và Slave thông qua bus hoặc theo mô hình kết nối điểm-điểm. Trong một hệ thống có nhiều kết nối bus, mỗi kết nối có thể đảm nhận vai trò Master hoặc Slave tùy theo hướng khởi tạo giao dịch. Hình 2.1. minh họa giao tiếp giữa BUS 0 và BUS 1, trong đó bus khởi tạo giao dịch đóng vai trò Master, còn bus nhận yêu cầu đóng vai trò Slave.



Hình 2.1. Giao diện giao tiếp và kết nối bus

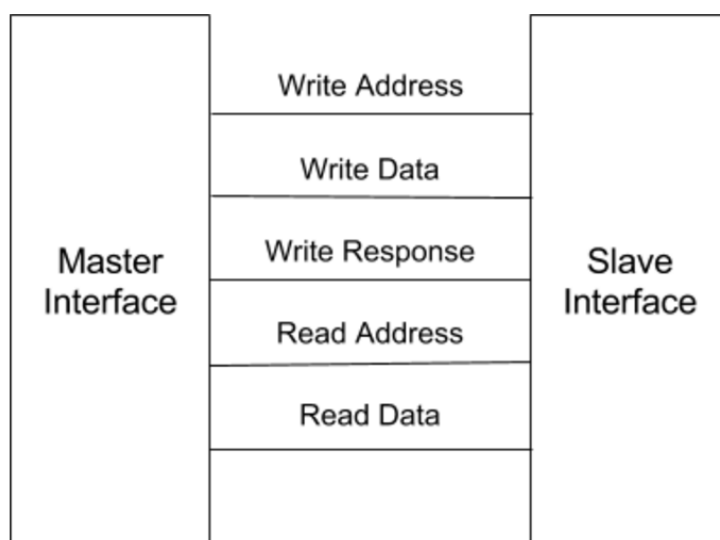
2.2.1.2 Cấu trúc bus AXI

Giao thức quy định cơ chế, cách thức hoạt động cho quá trình đọc và ghi dữ liệu. Hai hoạt động này được gọi chung là một truy cập. Mỗi lần truy cập được hiểu là một giao dịch.

AXI hoạt động dựa trên năm kênh độc lập, hình 2.2. mô tả hai khối Master và Slave được kết nối với nhau thông qua năm kênh này. Mỗi kênh đảm nhiệm một chức năng riêng. Mặc dù hoạt động độc lập, các kênh này vẫn phối hợp chặt chẽ để đảm bảo tính toàn vẹn và hiệu quả của giao dịch. Năm kênh này bao gồm:

- **Kênh địa chỉ ghi (Write Address Channel):** truyền thông tin địa chỉ và thông tin điều khiển liên quan đến quá trình ghi từ Master đến Slave.
- **Kênh dữ liệu ghi (Write Data Channel):** truyền dữ liệu ghi từ Master đến Slave.

- **Kênh phản hồi ghi (Write Response Channel):** truyền thông tin phản hồi cho một giao dịch ghi từ Slave đến Master sau khi hoàn thành giao dịch ghi.
- **Kênh địa chỉ đọc (Read Address Channel):** truyền thông tin địa chỉ và thông tin điều khiển của một giao dịch đọc từ Master đến Slave.
- **Kênh dữ liệu đọc (Read Data Channel):** truyền dữ liệu đọc và thông tin phản hồi của một giao dịch đọc từ Slave đến Master.



Hình 2.2. Các kênh giao tiếp giữa Master và Slave

Các tín hiệu trong mỗi kênh được giao thức AXI được quy định rõ ràng và được trình bày chi tiết trong các bảng từ bảng 2.1 đến bảng 2.5. Trong đó các tín hiệu ở kênh địa chỉ ghi được thể hiện ở bảng 2.1, các tín hiệu ở kênh dữ liệu ghi được thể hiện ở bảng 2.2, các tín hiệu ở kênh phản hồi ghi được thể hiện ở bảng 2.3, các tín hiệu ở kênh địa chỉ đọc được thể hiện ở bảng 2.4 và các tín hiệu ở kênh dữ liệu đọc được thể hiện ở bảng 2.5.

Bảng 2.1 bên dưới liệt kê các tín hiệu thuộc kênh địa chỉ ghi, được sử dụng bởi Master để gửi thông tin địa chỉ đến Slave, đồng thời cung cấp các thuộc tính bổ sung như độ dài của dữ liệu truyền, kích thước dữ liệu, loại truy cập, thuộc tính bảo mật, chất lượng dịch vụ và vùng định tuyến. Hầu hết các tín hiệu trong kênh này đều được xuất phát từ Master, ngoại trừ tín hiệu AWREADY được xuất phát từ Slave thông báo tình trạng sẵn sàng nhận thông tin địa chỉ gửi từ Master.

Bảng 2.1. Các tín hiệu của kênh địa chỉ ghi

TÍN HIỆU	NGUỒN
AWID	Master
AWADDR	Master
AWLEN	Master
AWSIZE	Master
AWBURST	Master
AWLOCK	Master
AWCACHE	Master
AWPROT	Master
AWQOS	Master
AWREGION	Master
AWUSER	Master
AWVALID	Master
AWREADY	Slave

Bảng 2.2 bên dưới mô tả các tín hiệu thuộc kênh dữ liệu ghi, trong đó Master truyền dữ liệu đến Slave. Các tín hiệu như WDATA,WSTRB,WLAST,WVALID đóng vai trò xác định nội dung, độ dài, dữ liệu cuối cùng của một giao dịch ghi và tín hiệu thông báo Master đã sẵn sàng truyền dữ liệu. Riêng đối với tín hiệu WREADY xuất phát từ SLAVE nhằm thông báo trạng thái sẵn sàng nhận dữ liệu ghi.

Bảng 2.2. Các tín hiệu của kênh dữ liệu ghi

TÍN HIỆU	NGUỒN
WID	Master
WDATA	Master
WSTRB	Master
WLAST	Master
WUSER	Master
WVALID	Master
WREADY	Slave

Bảng 2.3 trình bày các tín hiệu thuộc kênh phản hồi ghi trong giao thức AXI4. Đây là kênh phản hồi từ Slave về Master sau khi một giao dịch ghi được

thực hiện, nhằm thông báo dữ liệu ghi thành công hay xuất hiện lỗi. Các tín hiệu như BID, BRESP, BUSER và BVALID được xuất phát từ Slave, trong khi tín hiệu BREADY do Master điều khiển để thông báo tình trạng sẵn sàng nhận tín hiệu từ kênh phản hồi ghi.

Bảng 2.3. Các tín hiệu của kênh phản hồi ghi

TÍN HIỆU	NGUỒN
BID	Slave
BRESP	Slave
BUSER	Slave
BVALID	Slave
BREADY	Master

Bảng 2.4 mô tả các tín hiệu của kênh địa chỉ đọc. Đây là kênh chịu trách nhiệm truyền thông tin địa chỉ địa chỉ đọc cho các yêu cầu đọc từ Master đến Slave. Tất cả các tín hiệu trong kênh này đều được điều khiển bởi Master, phản ánh vai trò chủ động của Master trong việc khởi tạo các giao dịch đọc.

Bảng 2.4. Các tín hiệu của kênh địa chỉ đọc

TÍN HIỆU	NGUỒN
ARID	Master
ARADDR	Master
ARLEN	Master
ARLEN	Master
ARSIZE	Master
ARBURST	Master
ARLOCK	Master
ARCACHE	Master
ARPROT	Master
ARQOS	Master
ARREGION	Master
ARUSER	Master
ARVALID	Master
ARREADY	Master

Bảng 2.5 liệt kê các tín hiệu thuộc kênh dữ liệu đọc trong giao thức AXI4. Đây là kênh chịu trách nhiệm truyền dữ liệu từ Slave về Master sau khi yêu cầu đọc được thiết lập qua kênh địa chỉ đọc. Các tín hiệu như RID, RDATA, RRESP, RLAST, RUSER và RVALID đều được điều khiển bởi Slave. Trong khi đó, tín hiệu RREADY xuất phát từ Master để thông báo đã sẵn sàng nhận dữ liệu.

Bảng 2.5. Các tín hiệu của kênh dữ liệu đọc

TÍN HIỆU	NGUỒN
RID	Slave
RDATA	Slave
RRESP	Slave
RLAST	Slave
RUSER	Slave
RVALID	Slave
RREADY	Master

2.2.1.3 Giới thiệu về giao thức AMBA AXI4

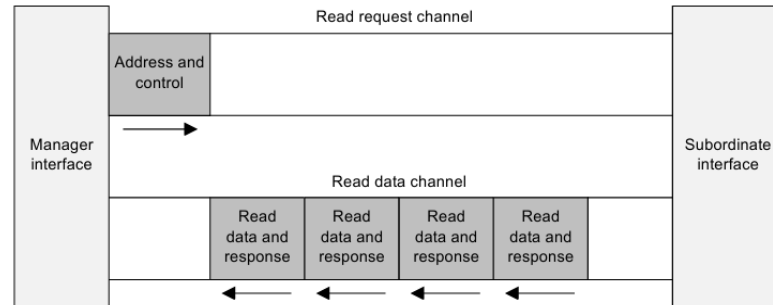
Giao thức AXI là một phần trong hệ thống chuẩn AMBA do hãng ARM phát triển từ những năm 1990, nhằm chuẩn hóa giao tiếp giữa các khối phần cứng trong vi mạch. Phiên bản AXI4 được giới thiệu vào năm 2010, như một cải tiến lớn so với AXI3 và các chuẩn trước đó.

Các đặc trưng nổi bật của giao thức AXI4 bao gồm:

- Tách riêng kênh đọc và kênh ghi, pha truyền địa chỉ và điều khiển với pha truyền dữ liệu.
- Hỗ trợ trao đổi dữ liệu không thẳng hàng và sử dụng tín hiệu write strobe để xác định byte nào trong dữ liệu là hợp lệ, từ đó nâng cao độ linh hoạt khi truy cập bộ nhớ.
- Cho phép chèn thêm các tầng thanh ghi giúp định thời của bus và hệ thống tốt hơn.
- Các giao dịch có thể hoàn thành không theo thứ tự.
- Hỗ trợ phát hành nhiều địa chỉ truy cập đồng thời.
- Sử dụng cơ chế truyền theo burst, chỉ cần phát địa chỉ bắt đầu cho toàn bộ chuỗi truyền, giúp giảm tải overhead cho hệ thống [9].

2.2.1.4 Các quy định về hoạt động

Trong giao thức AXI, mỗi truy cập ghi hoặc đọc đều được xem là một giao dịch. Đối với một giao dịch đọc, quá trình truyền dữ liệu sẽ diễn ra thông qua hai kênh: kênh địa chỉ đọc và kênh dữ liệu đọc. Hình 2.3. minh họa quá trình hoạt động của một giao dịch đọc trong giao thức AXI.

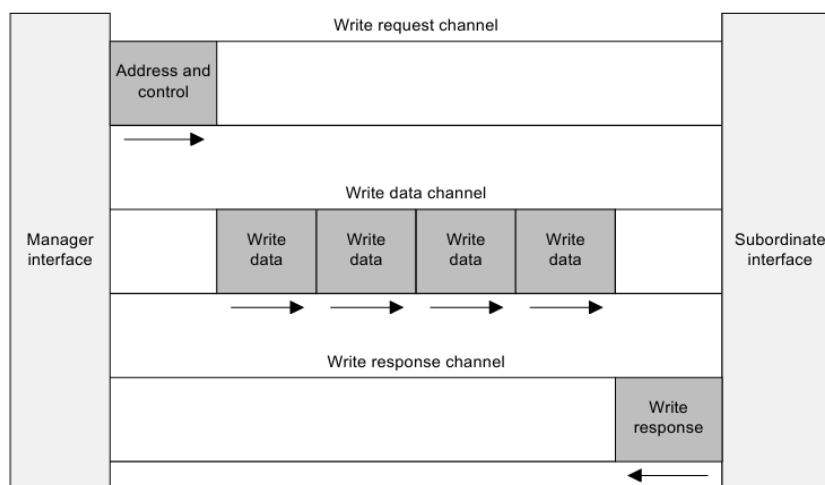


Hình 2.3. Hoạt động của giao dịch đọc

Một giao dịch đọc gồm hai bước chính:

- Master gửi địa chỉ cần đọc cùng với các thông tin điều khiển lên kênh địa chỉ đọc để khởi động quá trình đọc.
- Dữ liệu cùng với thông tin phản hồi sẽ được Slave truyền ngược lại cho Master thông qua kênh dữ liệu đọc. Số lượng dữ liệu được truyền sẽ phụ thuộc vào các thông tin điều khiển đã được gửi.

Một giao dịch ghi sẽ được thực hiện thông qua ba kênh: kênh địa chỉ ghi, kênh dữ liệu ghi và kênh phản hồi. Hình 2.4. minh họa trình tự hoạt động của một giao dịch ghi.



Hình 2.4. Hoạt động của giao dịch ghi

Một giao dịch ghi gồm ba bước:

- Master gửi địa chỉ ghi và thông tin điều khiển lên kênh địa chỉ ghi để khởi động quá trình ghi.
- Dữ liệu cần ghi được truyền từ Master đến Slave thông qua kênh dữ liệu ghi. Số lượng dữ liệu được xác định dựa trên thông tin điều khiển đã được gửi ở bước đầu.
- Sau khi hoàn tất dữ liệu ghi, Slave gửi tín hiệu phản hồi về cho Master thông qua kênh phản hồi ghi để xác nhận quá trình ghi đã hoàn tất.

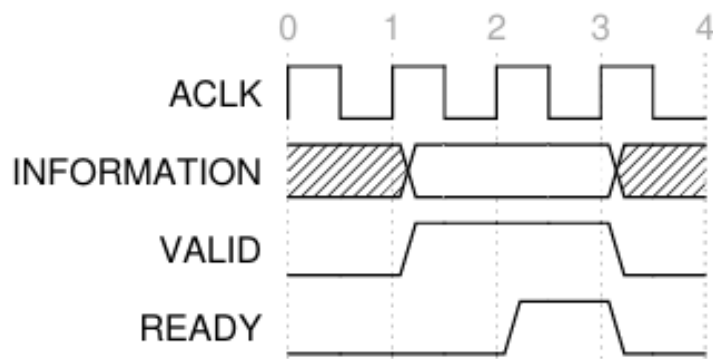
2.2.1.5 Cơ chế bắt tay của AXI4

Giao thức AXI hoạt động dựa trên cơ chế bắt tay hai chiều, trong đó quá trình truyền dữ liệu được điều phối thông qua cặp tín hiệu VALID và READY trên từng kênh truyền riêng biệt. Nguyên tắc vận hành cơ chế này là bên phát sẽ khởi tạo tín hiệu VALID tích cực khi có dữ liệu hợp lệ cần truyền. Tín hiệu được duy trì cho đến khi bên nhận phát ra tín hiệu READY tích cực, cho biết sẵn sàng tiếp nhận dữ liệu. Quá trình truyền dữ liệu chỉ được thực hiện tại thời điểm cả hai tín hiệu VALID và READY cùng đạt mức tích cực trên cạnh lên của xung nhịp. Tín hiệu READY từ phía nhận có thể xuất hiện trước, đồng thời hoặc sau tín hiệu VALID từ phía truyền. Điều này dẫn đến ba trường hợp khác nhau trong cơ chế bắt tay, thể hiện sự linh hoạt của giao thức trong việc đồng bộ hóa truyền nhận dữ liệu. Mỗi kênh chức năng trong AXI4 bao gồm các kênh địa chỉ ghi, dữ liệu ghi, phản hồi ghi, địa chỉ đọc và dữ liệu đọc đều được trang bị cặp tín hiệu VALID, READY độc lập, bảo đảm khả năng truyền song song và hiệu quả cao trong hệ thống. Các cặp tín hiệu bắt tay của các kênh được trình bày ở bảng 2.6.

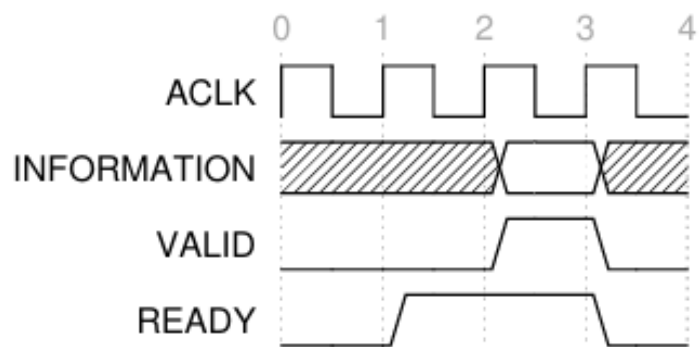
Bảng 2.6. Các cặp tín hiệu bắt tay trong giao thức AXI4

KÊNH	CẶP TÍN HIỆU BẮT TAY
Kênh địa chỉ ghi	AWVALID – AWREADY
Kênh dữ liệu ghi	WVALID – WREADY
Kênh phản hồi ghi	BVALID – BREADY
Kênh địa chỉ đọc	ARVALID – ARREADY
Kênh dữ liệu đọc	RVALID – RREADY

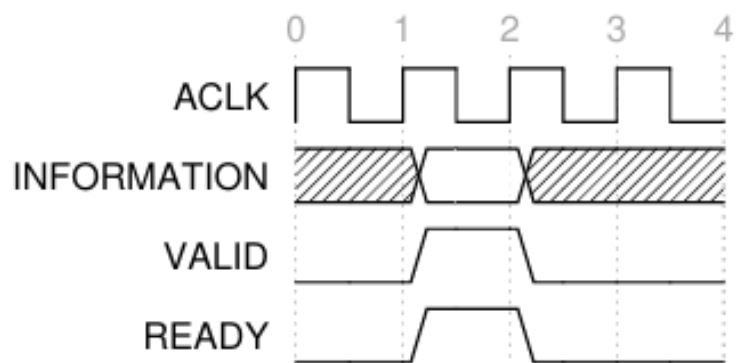
Trong quá trình truyền dữ liệu, một số tình huống phổ biến có thể xảy ra trong quá trình bắt tay: tín hiệu VALID tích cực trước tín hiệu READY được minh họa ở hình 2.5, tín hiệu VALID tích cực sau tín hiệu READY được thể hiện ở hình 2.6, và trường hợp cuối cùng là tín hiệu VALID tích cực cùng lúc với tín hiệu READY được thể hiện ở hình 2.7.



Hình 2.5. Tín hiệu VALID tích cực trước tín hiệu READY



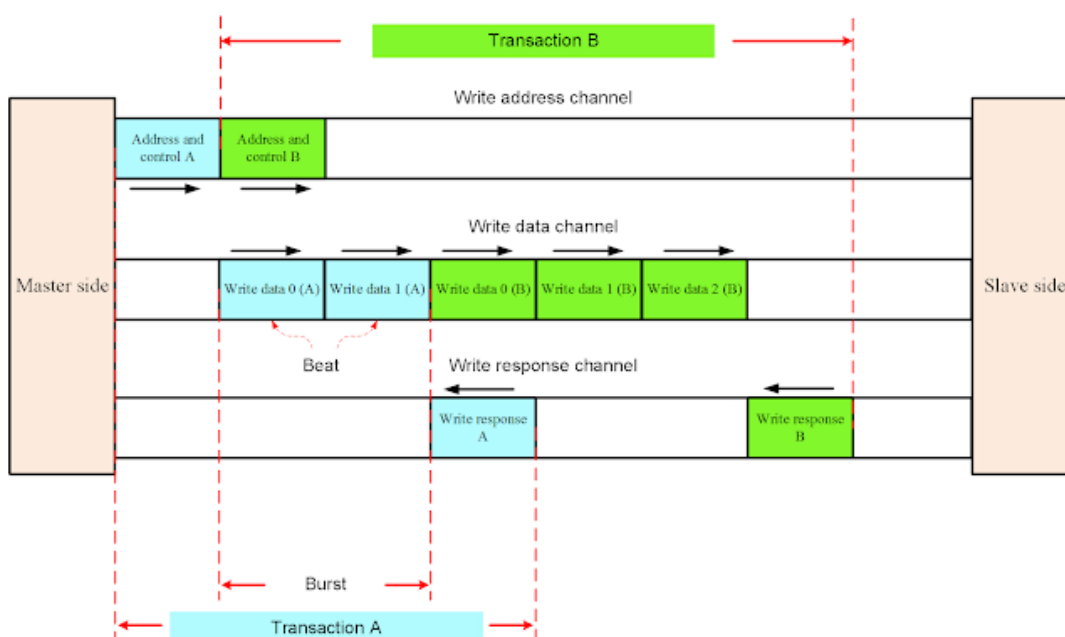
Hình 2.6. Tín hiệu VALID tích cực sau tín hiệu READY



Hình 2.7. Tín hiệu VALID tích cực cùng lúc với tín hiệu READY

2.2.1.6 Cơ chế burst của AXI4

Giao thức AXI4 hoạt động dựa trên cơ chế truyền dữ liệu theo kiểu burst. Cơ chế burst theo AXI4 được phân thành ba loại chính bao gồm: FIXED, INCR, và WRAP. Mỗi burst bao gồm một chuỗi các lần truyền dữ liệu (transfer) thuộc cùng một giao dịch, trong đó mỗi lần truyền dữ liệu được gọi là một beat. Một burst có thể chứa một hoặc nhiều beat, và thông tin điều khiển sẽ bao gồm các thuộc tính mô tả đặc điểm của burst trong mỗi giao dịch. Hình 2.8. minh họa cụ thể hoạt động của cơ chế burst.



Hình 2.8. Minh họa cơ chế burst

Loại burst trong giao thức AXI4 được xác định thông qua tín hiệu AxBURST. Giao thức hỗ trợ ba loại burst chính:

- **FIXED:** Địa chỉ của mỗi lần truyền dữ liệu trong một giao dịch đều giống nhau, tức là không thay đổi theo từng beat.
- **INCR:** Địa chỉ của mỗi lần truyền dữ liệu kế tiếp được tính bằng cách cộng thêm một giá trị cố định vào địa chỉ trước đó. Giá trị cộng thêm này được xác định bởi tín hiệu AxSIZE [9]. Công thức tính địa chỉ truy cập như sau:

$$\text{Aligned_Addr} = \text{INT} (\text{Start_Addr} / \text{Size}) * \text{Size} \quad (2.1)$$

$$\text{Address_N} = \text{Start_Address} + (\text{N} - 1) * \text{Size} \quad (2.2)$$

Với:

Start_Addr (= axaddr): Địa chỉ bắt đầu ghi dữ liệu

INT(x): Giá làm tròn xuống của x

N: Số lần truyền dữ liệu

Size: Kích thước mỗi transfer = 2^{Sizes}

- **WRAP:** Ban đầu địa chỉ của mỗi lần truyền dữ liệu sẽ được tính như kiểu INCR. Khi địa chỉ truy cập đạt đến địa chỉ cuộn, địa chỉ sẽ được trở về địa chỉ cuộn (wrap boundary) và tiếp tục truyền. Công thức tính địa chỉ truy cập như sau:

$$\text{Wrap_Boundary} = \text{INT}(\text{Start_Addr} / (\text{Size} * \text{Length})) * \text{Size} * \text{Length} \quad (2.3)$$

Nếu địa chỉ chưa vượt vùng wrap:

$$\text{Address_N} = \text{Aligned_Addr} + (N - 1) * \text{Size} \quad (2.4)$$

Nếu địa chỉ vượt biên và cần wrap lại:

$$\text{Address_N} = \text{Start_Addr} + ((N - 1) * \text{Size}) - (\text{Size} * \text{Length}) \quad (2.5)$$

Với Length: số lượng transfer = $\text{AxLEN} + 1$

Bảng 2.7. dưới đây thể hiện giá trị AxBURST tương ứng với cơ chế burst.

Bảng 2.7. Các loại burst tương ứng với giá trị AxBURST

AxBURST	Label	Meaning
0b00	FIXED	Fixed burst
0b01	INCR	Incrementing burst
0b10	WRAP	Wrapping burst
0b11	RESERVED	-

Độ dài một burst được quy định bởi tín hiệu AxLEN và được tính như sau:

$$\text{Độ dài burst (số beat)} = \text{AxLEN} + 1 \quad (2.6)$$

Số byte tối đa trong một lần truyền dữ liệu (hay còn gọi là một beat) được xác định bởi AxSIZE. Giá trị này quy định kích thước của mỗi beat và không được vượt quá độ rộng của bus dữ liệu (tức là độ rộng của các tín hiệu RDATA

hoặc WDATA). Chẳng hạn, nếu bus dữ liệu có độ rộng 32 byte, thì giá trị của AxSIZE[2:0] phải nhỏ hơn hoặc bằng 5. Giá trị của AxSIZE được quy định ở bảng 2.8. dưới đây.

Bảng 2.8. Kích thước dữ liệu tương ứng với giá trị AxLEN

AxSIZE	Label	Meaning
0b000	1	1 byte per transfer
0b001	2	2 bytes per transfer
0b010	4	4 bytes per transfer
0b011	8	8 bytes per transfer
0b100	16	16 bytes per transfer
0b101	32	32 bytes per transfer
0b110	64	64 bytes per transfer
0b111	128	128 bytes per transfer

2.2.2 Tổng quan về giao thức AMBA APB

2.2.2.1 Giới thiệu chung về giao thức AMBA APB

Giao thức APB là giao thức bus thuộc chuẩn họ AMBA do hãng ARM phát triển, được thiết kế nhằm kết nối các thiết bị ngoại vi tốc độ thấp trong hệ thống trên chip. Khác với các bus tốc độ cao AXI, giao thức APB có cấu trúc đơn giản, không hỗ trợ pipeline hay truyền song song, từ đó giúp giảm độ phức tạp phần cứng và tiết kiệm năng lượng. APB thường được sử dụng cho các ngoại vi như UART, GPIO, TIMER, bộ điều khiển ngắt và các thanh ghi cấu hình. Trong kiến trúc chip, APB thường được đặt phía sau các bus tốc độ cao thông qua các cầu chuyển đổi, đảm bảo sự cân bằng giữa hiệu năng và công suất tiêu thụ. Hiện nay, hãng ARM vẫn đang phát triển thêm nhiều phiên bản APB để đáp ứng được việc kiến trúc SoC ngày càng phức tạp.

2.2.2.2 Tín hiệu của giao thức APB

Giao thức APB sử dụng tập tín hiệu đơn giản nhằm thực hiện các giao dịch đọc và ghi giữa Master và Slave. Các tín hiệu chính của giao thức APB được thể hiện ở bảng 2.9. dưới đây.

Bảng 2.9. Tín hiệu của giao thức APB

TÍN HIỆU	MÔ TẢ
PCLK	Tín hiệu xung nhịp
PRESETn	Tín hiệu reset mức tích cực thấp dùng để khởi tạo lại hệ thống
PADDR	Địa chỉ của thanh ghi hoặc ngoại vi mà Master muốn truy cập
PSELx	Tín hiệu chọn Slave, mỗi Slave tương ứng với một tín hiệu PSEL riêng
PENABLE	Tín hiệu cho biết giai đoạn truy cập
PWRITE	Tín hiệu xác định loại giao dịch ghi (PWRITE = 1) hoặc giao dịch đọc (PWRITE = 0)
PWDATA	Dữ liệu ghi từ Master đến Slave
PRDATA	Dữ liệu đọc từ Slave về Master
PREADY	Tín hiệu phản hồi từ Slave, cho biết Slave đã sẵn sàng hoàn tất giao dịch
PSLVERR	Tín hiệu báo lỗi, được kích hoạt khi xảy ra lỗi trong quá trình truyền

2.2.2.3 Các quy định về hoạt động

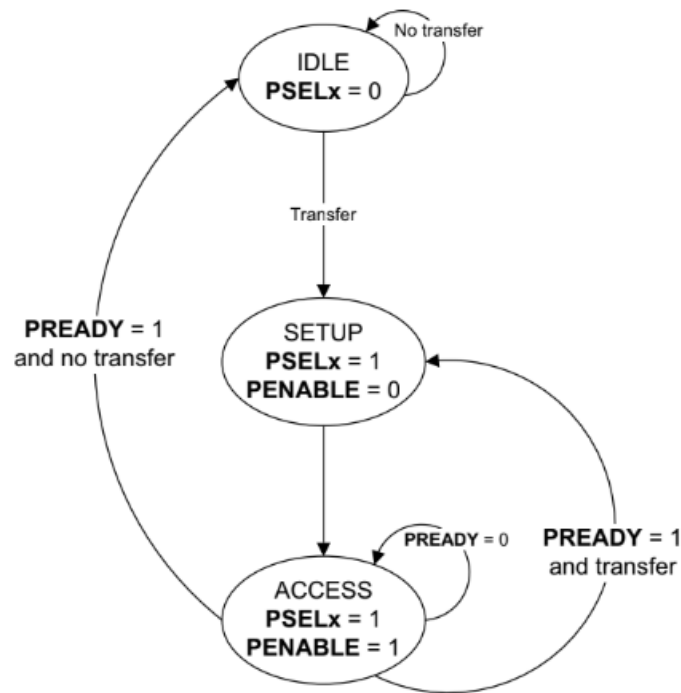
Các tín hiệu của giao thức APB hoạt động theo xung cạnh lên PCLK, hoạt động của APB được chia thành ba trạng thái chính như hình 2.9. mô tả, gồm các trạng thái:

+ Trạng thái IDLE: bus APB ở trạng thái không hoạt động và không có giao dịch nào thực hiện. Ở trạng thái này, PSEL không tích cực, cho biết không có Slave nào được chọn.

+ Trạng thái SETUP: Khi có yêu cầu truy cập, tín hiệu PSEL tích cực. Các tín hiệu PADDR, PWRITE, PWDATA (giao dịch ghi), PRDATA (giao dịch đọc) được giữ ổn định cho đến khi hoàn thành giao dịch.

+ Trạng thái ACCESS: Tín hiệu PENABLE tích cực ở chu kỳ xung nhịp tiếp theo. Giao dịch hoàn tất khi tín hiệu PREADY tích cực. Nếu tín hiệu PREADY ở mức thấp, trạng thái ACCESS sẽ được kéo dài cho đến khi

PREADY tích cực, sau đó giao thức quay về trạng thái IDLE để chuẩn bị một giao dịch mới [10].



Hình 2.9. Máy trạng thái mô tả hoạt động giao thức APB

2.3 TỔNG QUAN VỀ PHƯƠNG THỨC XÁC MINH UVM

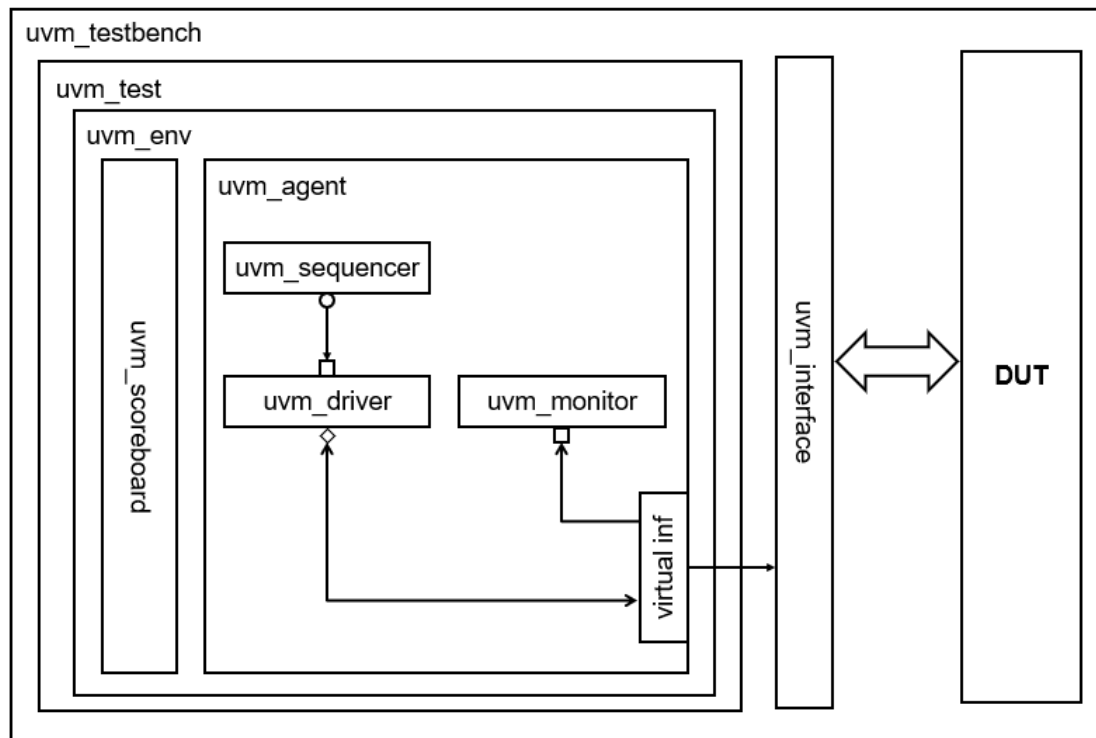
UVM là phương pháp xác minh chức năng tiêu chuẩn do tổ chức Accellera phát triển, được xây dựng dựa trên nguyên tắc hợp tác giữa các nhà cung cấp công cụ EDA và cộng đồng người dùng. UVM kế thừa và hợp nhất các nền tảng xác minh đã được kiểm chứng trước đó như OVM, VMM, eRM và AVM. Đồng thời, nhà phát triển tích hợp thêm các cơ chế và công nghệ mới như Resource Database, TLM-2.0 và Phasing, nhằm hình thành một phương pháp xác minh thống nhất, linh hoạt và có khả năng mở rộng cao [11].

Với OVM làm nền tảng cốt lõi, UVM phản ánh nhiều năm kinh nghiệm trong thiết kế testbench theo hướng đối tượng và xác minh các hệ thống số phức tạp. Phương thức này hỗ trợ xây dựng các môi trường xác minh có khả năng năng mở rộng, tính tái sử dụng và tương thích cao, đáp ứng được nhu cầu xác minh các thiết kế chip ngày càng lớn và phức tạp trong hệ thống trên chip hiện đại. Việc UVM được chuẩn hóa thành IEEE 1800.2 vào năm 2017 đã khẳng định

vị thế của UVM như một tiêu chuẩn chung trong lĩnh vực xác minh chức năng, đồng thời tạo điều kiện cho việc áp dụng thống nhất UVM trên nhiều nền tảng thực thi khác nhau như mô phỏng, tăng tốc phần cứng, emulation và FPGA prototyping [11].

2.3.1 Cấu trúc UVM Testbench

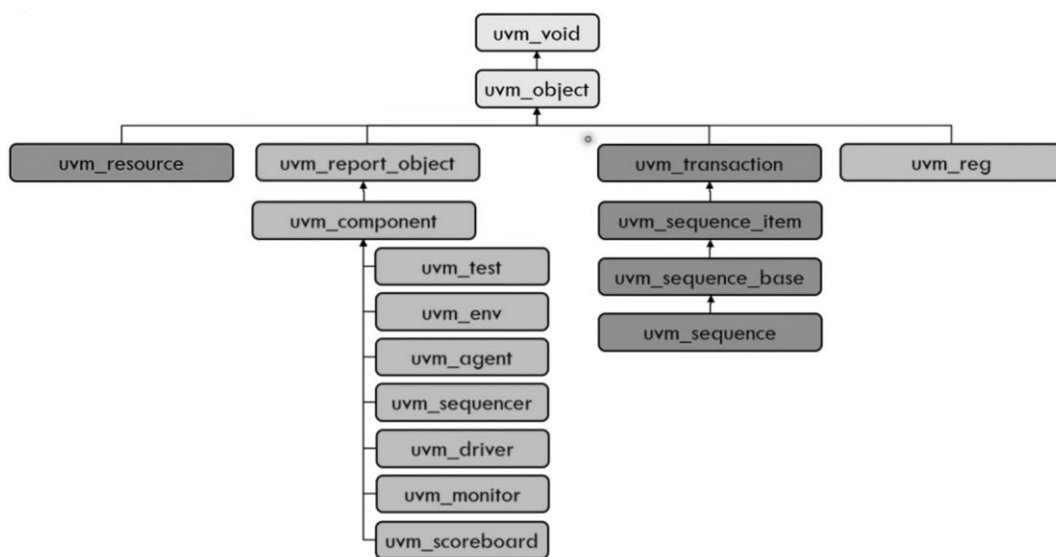
UVM cung cấp các thư viện lớp cần thiết để xây dựng các môi trường xác minh có tính linh động và tái sử dụng. Các thư viện này bao gồm các tiện ích về cấu hình, giao dịch và giao tiếp, cho phép người sử dụng tổ chức các thành phần xác minh theo một cấu trúc cụ thể. Nhờ đó, các thành phần trong testbench có thể trao đổi dữ liệu một cách đồng bộ và hiệu quả. Hình 2.10. dưới đây minh họa cấu trúc UVM testbench điển hình.



Hình 2.10. Cấu trúc điển hình của UVM

Trong một UVM testbench, các thành phần được tạo ra từ lớp cơ sở `uvm_component` sẽ xây dựng nên một cấu trúc phân cấp ổn định, được duy trì xuyên suốt quá trình mô phỏng. Các thành phần này sẽ tồn tại cho đến khi kết thúc mô phỏng bao gồm environment, agent, monitor, driver và scoreboard. Chúng tạo nên cấu trúc tổng thể và điều khiển hành vi của testbench, bao gồm cấu hình, sinh kích thích (stimulus), kiểm tra dữ liệu, báo cáo kết quả kiểm thử.

Ngược lại, các đối tượng như `uvm_sequence`, `uvm_sequence_item` và nhiều đối tượng khác mang tính tạm thời. Chúng được tạo động để định nghĩa các chuỗi kiểm thử cụ thể, được thực thi trong quá trình mô phỏng và sau đó bị hủy khi hoàn thành nhiệm vụ. Các sequence chứa các kịch bản kiểm thử và các thông tin cần thiết. Đồng thời, các đối tượng `uvm_sequencer` và `uvm_driver` giao tiếp với DUT thông qua interface. Hình 2.11. thể hiện phân cấp UVM.



Hình 2.11. Phân cấp UVM

2.3.2 Lớp cấu trúc UVM Testbench

2.3.2.1 UVM Testbench Top

UVM Testbench là khối bao quanh thiết kế phần cứng cần kiểm tra (DUT), chịu trách nhiệm khởi tạo DUT, lớp UVM Test và cấu hình kết nối giữa các thành phần xác minh. UVM Testbench chỉ cần biên dịch một lần nhưng có thể thực thi với nhiều kịch bản kiểm thử khác nhau nhờ cơ chế khởi tạo tự động của lớp UVM Test trong thời gian mô phỏng [12].

2.3.2.2 UVM Test

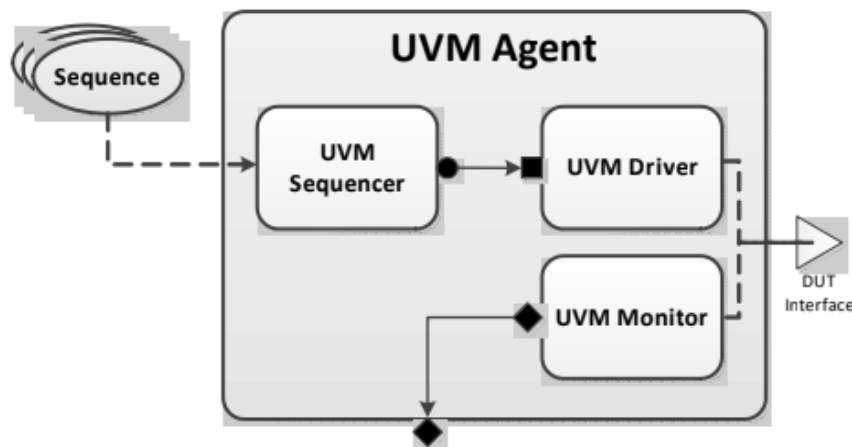
UVM Test là thành phần UVM ở mức cao nhất trong testbench. Thành phần này có nhiệm vụ khởi tạo môi trường xác minh, cấu hình các tham số thông qua cơ chế Factory hoặc cơ sở dữ liệu cấu hình, đồng thời điều khiển việc sinh kích thích bằng cách kích hoạt các sequence để tác động lên DUT. Thông thường, một test cơ sở được xây dựng trước, sau đó các test cụ thể sẽ kế thừa và điều chỉnh cấu hình hoặc lựa chọn các sequence khác nhau.

2.3.2.3 UVM Environment

UVM Environment là một thành phần phân cấp dùng để nhóm các khối xác minh có liên quan với nhau, chẳng hạn như agent, scoreboard hoặc các environment con. Environment ở mức cao nhất bao gồm toàn bộ các thành phần xác minh hướng tới DUT, đặc biệt trong các hệ thống phức tạp như SoC, nơi mỗi IP thường được tổ chức thành một environment riêng.

2.3.2.4 UVM Agent

UVM Agent đại diện cho một giao diện cụ thể của DUT và bao gồm các thành phần như sequencer, driver và monitor. Agent có thể hoạt động ở chế độ chủ động (active), khi có khả năng sinh và điều khiển kích thích, hoặc ở chế độ thụ động (passive), khi chỉ thực hiện giám sát giao diện [12]. Hình 2.12. dưới đây mô tả hoạt động truyền thông tin từ UVM Test vào UVM Agent.



Hình 2.12. UVM Agent

2.3.2.5 UVM Sequencer

UVM Sequencer có nhiệm vụ điều phối luồng giao dịch, quản lý và phân xử các transaction được sinh ra từ một hoặc nhiều sequence khác nhau trước khi chuyển đến driver.

2.3.2.6 UVM Sequence

UVM Sequence là đối tượng sinh kích thích, không thuộc cấu trúc phân cấp của component. Sequence định nghĩa hành vi sinh transaction và có thể tồn tại trong thời gian ngắn hoặc xuyên suốt mô phỏng. Các sequence được tổ chức theo dạng phân cấp, trong đó sequence cha có thể gọi các sequence con [12].

2.3.2.7 UVM Driver

UVM Driver nhận các transaction từ sequencer và chuyển đổi chúng từ mức trừu tượng sang mức tín hiệu chân để tác động trực tiếp lên giao diện DUT. Driver đóng vai trò cầu nối giữa mức giao dịch và mức phần cứng.

2.3.2.8 UVM Monitor

UVM Monitor có nhiệm vụ quan sát các tín hiệu trên giao diện DUT, thu thập thông tin và chuyển đổi chúng thành các transaction gửi đến các thành phần phân tích như scoreboard hoặc coverage thông qua cổng phân tích TLM.

2.3.2.9 UVM Scoreboard

UVM Scoreboard thực hiện chức năng kiểm tra tính đúng đắn của DUT bằng cách so sánh dữ liệu đầu ra thực tế với dữ liệu mong đợi được tạo ra từ mô hình tham chiếu, qua đó đánh giá hành vi của thiết kế.

2.4 CƠ CHẾ UVM

2.4.1 Cơ chế Pha

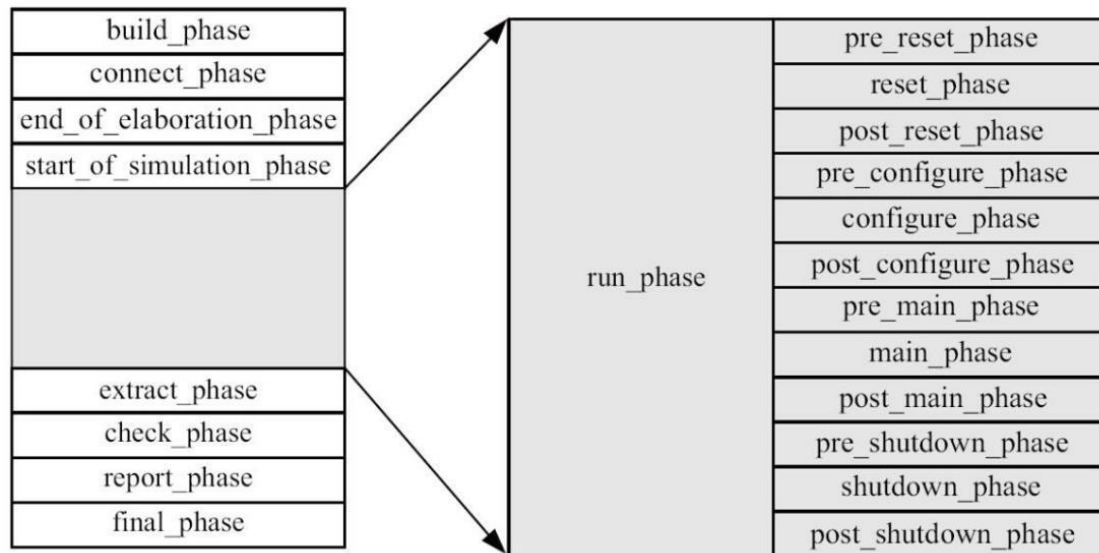
Để đảm bảo luồng thực thi của testbench được tổ chức chặt chẽ và nhất quán, UVM sử dụng cơ chế pha nhằm điều phối và sắp xếp các hoạt động chính diễn ra trong suốt quá trình mô phỏng. Các pha được chia thành ba nhóm:

- Pha khởi tạo: Pha sẽ thực hiện công đoạn từ `build_phase` đến `start_of_simulation_phase`, hoạt động chủ yếu là xây dựng cấu trúc, cấu hình và kết nối các thành phần trong UVM. Vì pha này là các function nên không tiêu tốn thời gian mô phỏng.

- Pha thực thi: giai đoạn tiêu tốn thời gian mô phỏng, trong đó các tệp kiểm tra được thực thi trên testbench. Hình 2.13. bên dưới cho thấy các giai đoạn của pha thực thi bao gồm rất nhiều công đoạn và có hai cách để thực hiện pha thực thi: thông qua chuỗi các công đoạn từ `pre_reset_phase` đến `post_shutdown_phase` hoặc thông qua công đoạn `run_phase`. Hai cách này có ý nghĩa tương đương và được thi song song, do đó khi xây dựng môi trường kiểm thử chỉ chọn một cách.

- Pha kết thúc: Pha sẽ thực hiện công đoạn từ `extract_phase` đến `final_phase`, thực hiện thu thập kết quả, tổng hợp và báo cáo dữ liệu kiểm thử.

Lớp cơ sở `uvm_component` cung cấp phương thức ảo tương ứng với từng pha, cho phép người sử dụng hiện thực hành vi của mỗi thành phần theo đúng vai trò của chúng trong môi trường kiểm thử. Việc tuân thủ cơ chế phasing không chỉ giúp các thành phần xác minh được phát triển một cách độc lập, mà còn đảm bảo khả năng phối hợp và tương tác chính xác giữa chúng, nhờ vào sự thống nhất về chức năng và trình tự hoạt động trong từng pha của quy trình xác minh [11].



Hình 2.13. Cơ chế pha trong UVM

2.4.2 Cơ chế Factory

Cơ chế Factory được sử dụng để tạo và quản lý các đối tượng trong môi trường kiểm thử một cách linh hoạt, thông qua một cơ chế trung gian thay vì khởi tạo trực tiếp theo lớp cụ thể. Cách tiếp cận này giúp tách biệt cấu trúc testbench khỏi kiểu đối tượng được sử dụng, cho phép thay thế hoặc mở rộng chức năng của các thành phần mà không cần sửa đổi mã nguồn gốc.

Để sử dụng factory, các lớp phải được đăng ký với factory thông qua các macro chuẩn của UVM, điển hình là `uvm_component_utils` đối với component và `uvm_object_utils` đối với object. Nhờ khả năng thay đổi hành vi thành phần một cách linh hoạt, cơ chế factory góp phần nâng cao tính tái sử dụng, khả năng mở rộng và đơn giản hóa quá trình phát triển, bảo trì môi trường, đặc biệt là môi trường dùng để kiểm thử các thiết kế quy mô lớn, yêu cầu nhiều cách kiểm thử.

2.4.3 Cơ chế Config_db

Cơ chế cơ sở dữ liệu cấu hình (config_db) được sử dụng để truyền tham số cấu hình giữa các thành phần trong môi trường kiểm thử một cách linh hoạt và có hệ thống. Cơ chế này cho phép thiết lập các giá trị cấu hình từ mức cao như test hoặc environment, sau đó tự động phân phối xuống các thành phần con như agent, driver hoặc monitor mà không cần kết nối trực tiếp giữa các đối tượng. Hai phương thức hỗ trợ trong cơ chế Config_db là set() và get(). Phương thức set() đảm nhận hỗ trợ hoạt động gửi thông tin config và phương thức get() để lấy thông tin đối tượng config.

2.4.4 Chức năng độ bao phủ và độ bao phủ chức năng

Trong quá trình xác minh thiết kế số, độ bao phủ hỗ trợ xác định các nhánh logic, số khối, cấu trúc đoạn mã trong thiết kế đã thực hiện. Trong phương thức xác minh UVM, độ bao phủ là một cơ sở quan trọng để đánh giá chất lượng của môi trường kiểm thử và mức độ tin cậy của kết quả xác minh.

Độ bao phủ chức năng hỗ trợ xác định chức năng của thiết kế đã thực hiện trong quá trình kiểm thử. Khác với độ bao phủ, độ bao phủ chức năng phản ánh trực tiếp mức độ đáp ứng yêu cầu thiết kế.

CHƯƠNG 3

THIẾT KẾ HỆ THỐNG

3.1 YÊU CẦU HỆ THỐNG

Thiết kế bộ chuyển đổi giao thức AXI4 sang APB4 sử dụng ngôn ngữ mô tả Verilog cần đạt yêu cầu:

- Bộ chuyển đổi giao thức chuyển đổi các tín hiệu liên quan đảm bảo tuân thủ đặc tả kỹ thuật do hãng ARM đề ra.
- Thu thập các thông số tài nguyên được đánh giá trên mô phỏng phần cứng và đa dạng tần số xung nhịp, qua đó đánh giá hiệu năng của bộ chuyển đổi.

Bên cạnh đó, thiết kế môi trường UVM kiểm thử bộ chuyển đổi phải đạt các yêu cầu:

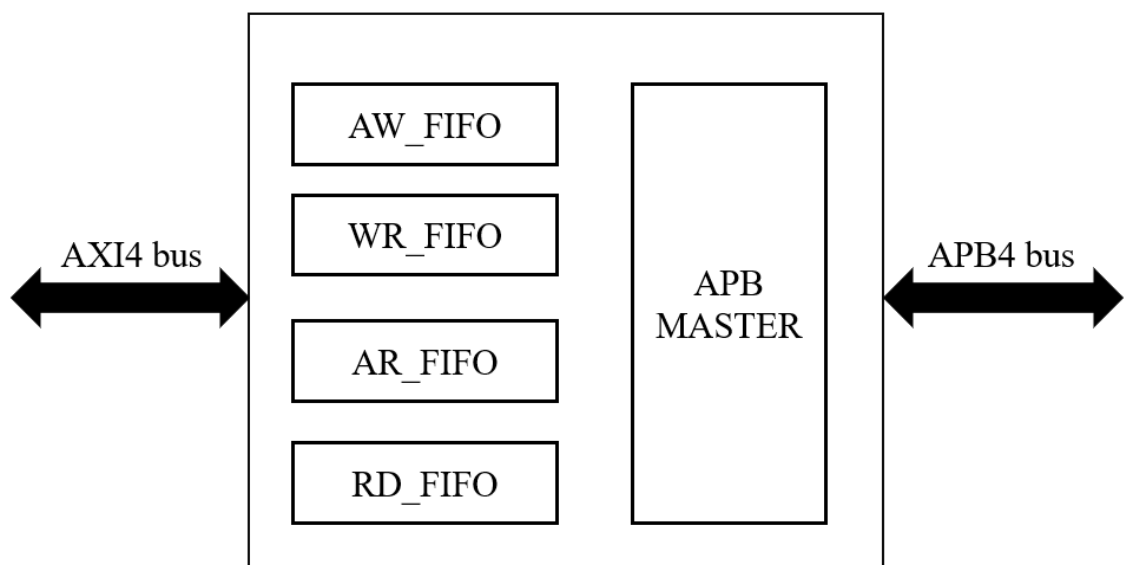
- Xây dựng môi trường UVM kiểm tra việc chuyển đổi giao thức của quá trình đọc và quá trình ghi theo ba cơ chế truyền dữ liệu: FIXED, INCR, WRAP với nhiều trường hợp độ dài truyền, kích thước dữ liệu và có lỗi phát sinh.
- Sử dụng kỹ thuật Constraint-Random cho phép sinh ra các trường hợp ngẫu nhiên có hoặc không có ràng buộc cho dữ liệu đầu vào.
- Sử dụng Assertion và Clocking Block để ràng buộc và kiểm tra các tín hiệu giao thức hoạt động đúng theo đặc tả hãng ARM.
- Các chỉ số độ bao phủ và độ bao phủ chức năng phải được áp dụng nhằm đánh giá mức độ đầy đủ của quá trình kiểm thử.

3.2 THIẾT KẾ BỘ CHUYỂN ĐỔI AXI4 SANG APB4

Hình 3.1 mô tả kiến trúc tổng thể của bộ chuyển đổi giao thức AXI4 sang APB4 được thiết kế cho hệ thống tích hợp ba ngoại vi được giao tiếp qua APB. Thiết kế được chia thành hai khối chức năng chính: khối đệm giao tiếp AXI4 và khối APB Master. Ở phía AXI4, bốn FIFO bất đồng bộ được sử dụng để tiếp

nhận và lưu trữ tạm thời dữ liệu từ các kênh truyền độc lập của giao thức AXI4, bao gồm: AW_FIFO cho kênh địa chỉ ghi, WR_FIFO cho kênh dữ liệu ghi, AR_FIFO cho kênh địa chỉ đọc và RD_FIFO cho kênh dữ liệu đọc. Việc tách riêng từng kênh vào một FIFO chuyên biệt cho phép duy trì tính toàn vẹn dữ liệu, đồng thời giảm thiểu hiện tượng nghẽn kênh khi tốc độ xử lý giữa hai giao thức không đồng bộ với nhau. Bên cạnh đó, các FIFO được thiết kế theo hướng tham số hóa, cho phép cấu hình độ sâu tùy theo yêu cầu băng thông và độ trễ của từng ứng dụng cụ thể.

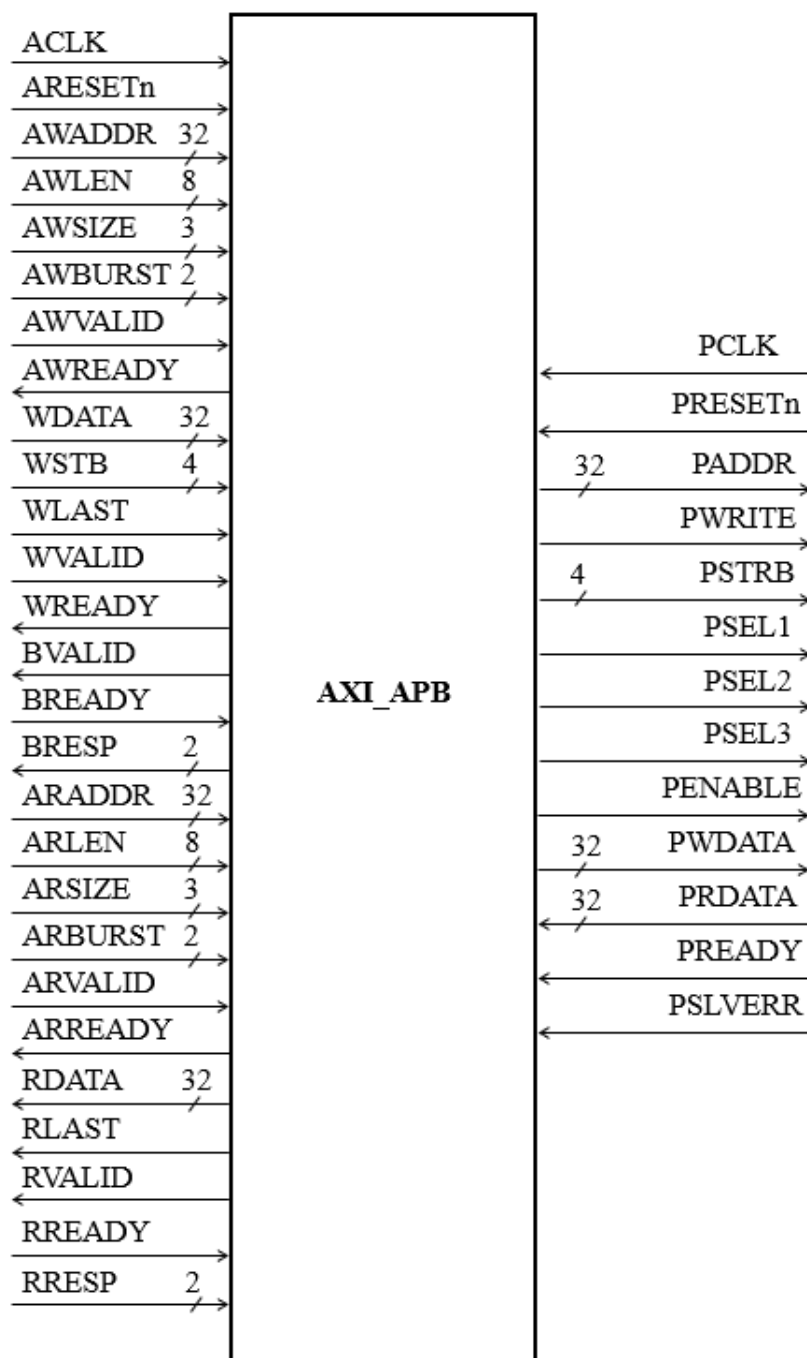
Khối APB Master đóng vai trò trung tâm trong quá trình chuyển đổi giao thức và quản lý truy cập đến ba thiết bị ngoại vi APB. Khối này chịu trách nhiệm đọc, ghi và giải mã các yêu cầu lưu trong các FIFO phía AXI4, tích hợp bộ giải mã địa chỉ và bộ phân xử để xác định Slave đích từ địa chỉ AXI4, từ đó tạo ra các tín hiệu điều khiển tương ứng để thực hiện giao dịch trên bus APB4. Kiến trúc tách biệt giữa miền AXI4 và miền APB4 giúp đảm bảo mỗi giao thức được xử lý theo đúng đặc tả thời gian và trình tự của nó. Nhờ đó, bộ chuyển đổi có thể hoạt động ổn định, hỗ trợ truyền dữ liệu chính xác và phù hợp với các hệ thống SoC đa miền xung nhịp. Toàn bộ các tín hiệu của bus AXI4 và APB4 đều được tích hợp đầy đủ theo tiêu chuẩn giao thức, đảm bảo khả năng tương thích cao với các IP lõi và thiết kế hệ thống phía sau.



Hình 3.1. Sơ đồ khối của cầu chuyển đổi

Tất cả các FIFO bất đồng bộ sử dụng hai xung nhịp và hai tín hiệu khởi tạo của giao thức AXI4 và APB4, đảm bảo việc đồng bộ hóa và truyền dữ liệu an toàn giữa hai miền. Ngược lại, khối APB Master chỉ hoạt động với một xung nhịp và một tín hiệu khởi tạo duy nhất thuộc miền APB4.

Hình 3.2. thể hiện sơ đồ tín hiệu kết nối được sử dụng cho bộ chuyển đổi giao thức AXI4 sang APB4.



Hình 3.2. Sơ đồ tín hiệu kết nối của khối chính

Dưới đây là bảng 3.1. mô tả các tín hiệu kết nối sử dụng cho bộ chuyển đổi giao thức AXI4 sang APB4.

Bảng 3.1. Các tín hiệu kết nối của bộ chuyển đổi AXI4 sang APB4

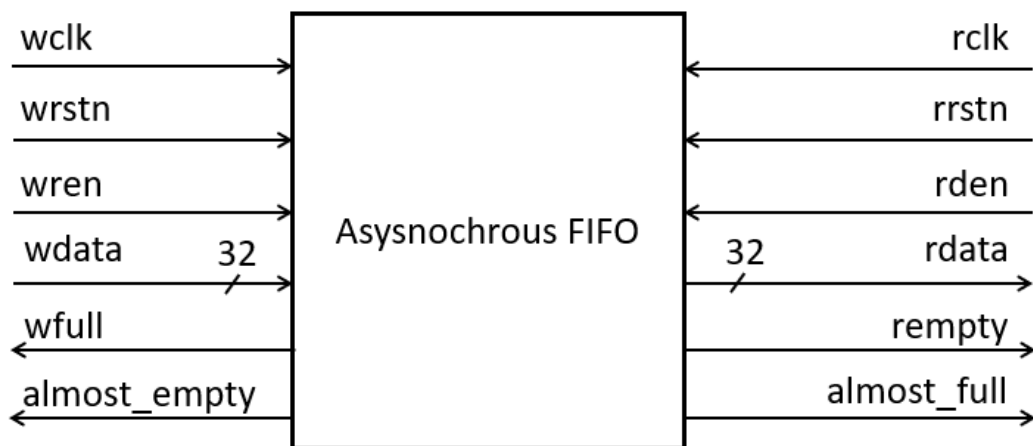
TÍN HIỆU	SỐ BIT	PHÂN LOẠI	MÔ TẢ
Các tín hiệu xung điều khiển			
ACLK	1	Ngõ vào	Tín hiệu xung nhịp đồng bộ hệ thống
ARESETn	1	Ngõ vào	Tín hiệu reset tích cực mức thấp để khởi tạo lại hệ thống
Kênh địa chỉ ghi của giao thức AXI4			
AWADDR	32	Ngõ vào	Địa chỉ bắt đầu transfer của giao dịch ghi
AWLEN	8	Ngõ vào	Số transfer của burst ghi
AWSIZE	3	Ngõ vào	Kích thước chung của một transfer trong một burst ghi
AWBURST	2	Ngõ vào	Loại burst ghi
AWVALID	1	Ngõ vào	Tín hiệu thông báo rằng địa chỉ ghi hợp lệ
AWREADY	1	Ngõ ra	Cho biết Slave đã sẵn sàng nhận địa chỉ
Kênh dữ liệu ghi của giao thức AXI4			
WDATA	32	Ngõ vào	Dữ liệu ghi
WSTRB	4	Ngõ vào	Tín hiệu cho phép truy cập byte dữ liệu ghi
WLAST	1	Ngõ vào	Tín hiệu thông báo transfer cuối cùng trong burst ghi
WVALID	1	Ngõ vào	Tín hiệu thông báo thông tin hợp lệ
WREADY	1	Ngõ ra	Tín hiệu Slave sẵn sàng nhận dữ liệu ghi
Kênh phản hồi ghi của giao thức AXI4			
BVALID	1	Ngõ vào	Thông báo một phản hồi hợp lệ
BREADY	1	Ngõ ra	Cho biết Master sẵn sàng nhận phản hồi

BRESP	2	Ngõ ra	Tín hiệu thông báo trạng thái giao dịch ghi
Kênh địa chỉ đọc của giao thức AXI4			
ARADDR	32	Ngõ vào	Địa chỉ bắt đầu transfer trong giao dịch đọc
ARLEN	8	Ngõ vào	Số transfer của burst đọc
ARSIZE	3	Ngõ vào	Kích thước chung của một transfer trong một burst đọc
ARBURST	2	Ngõ vào	Loại burst đọc
ARVALID	1	Ngõ vào	Tín hiệu thông báo rằng địa chỉ đọc hợp lệ
ARREADY	1	Ngõ ra	Cho biết Slave đã sẵn sàng nhận địa chỉ và tín hiệu liên quan
Kênh dữ liệu đọc của giao thức AXI4			
RDATA	32	Ngõ ra	Dữ liệu đọc
RLAST	1	Ngõ ra	Tín hiệu thông báo transfer cuối cùng trong burst đọc
RVALID	1	Ngõ ra	Tín hiệu thông báo dữ liệu đọc hợp lệ
RREADY	1	Ngõ vào	Tín hiệu cho biết Master sẵn sàng nhận dữ liệu đọc
RRESP	2	Ngõ ra	Tín hiệu thông báo trạng thái transfer đọc
Giao thức APB			
PCLK	1	Ngõ vào	Tín hiệu xung nhịp của giao thức APB
PRESETn	1	Ngõ vào	Tín hiệu reset tích cực mức thấp để khởi tạo lại hệ thống
PSTRB	4	Ngõ ra	Tín hiệu cho phép truy cập byte dữ liệu ghi
PADDR	32	Ngõ ra	Tín hiệu địa chỉ chứa thông tin địa chỉ ghi hoặc đọc
PSEL1	1	Ngõ ra	Tín hiệu thông báo Slave 1 được chọn
PSEL2	1	Ngõ ra	Tín hiệu thông báo Slave 2 được chọn

PSEL3	1	Ngõ ra	Tín hiệu thông báo Slave 3 được chọn
PENABLE	1	Ngõ ra	Tín hiệu cho phép đọc hoặc ghi dữ liệu
PWRITE	1	Ngõ ra	Tín hiệu cho phép đọc khi được tích cực
PWDATA	1	Ngõ ra	Dữ liệu ghi
PSTRB	4	Ngõ ra	Tín hiệu cho phép truy cập byte dữ liệu ghi
PREADY	1	Ngõ vào	Tín hiệu cho biết Master sẵn sàng nhận dữ liệu đọc
PRDATA	32	Ngõ vào	Dữ liệu đọc
PSLVERR	1	Ngõ vào	Tín hiệu có lỗi phát sinh trong quá trình đọc hoặc ghi

3.2.1 Khối Asynchronous FIFO

Khối Asynchronous FIFO đảm nhiệm chức năng lưu trữ tạm thời dữ liệu từ các kênh truyền độc lập của giao thức AXI4. Về nguyên tắc hoạt động, dữ liệu chỉ được ghi vào FIFO khi giao thức bắt tay (handshake) giữa tín hiệu VALID và READY của kênh tương ứng được thiết lập, qua đó đảm bảo tính toàn vẹn và tuần tự của luồng dữ liệu. Đặc biệt, cả hai thông số độ sâu (depth) và độ rộng (width) của FIFO đều được tham số hóa linh hoạt, cho phép tùy chỉnh để tối ưu hóa hiệu năng và tài nguyên cho từng mục đích cụ thể. Hình 3.2. mô tả sơ đồ các ngõ vào và ngõ ra của khối Asynchronous FIFO.



Hình 3.3. Sơ đồ tín hiệu kết nối của khối Asynchronous FIFO

Bảng 3.2. dưới đây cung cấp một mô tả chi tiết và đầy đủ về chức năng của từng tín hiệu.

Bảng 3.2. Chức năng của các tín hiệu trong khối Asynchronous FIFO

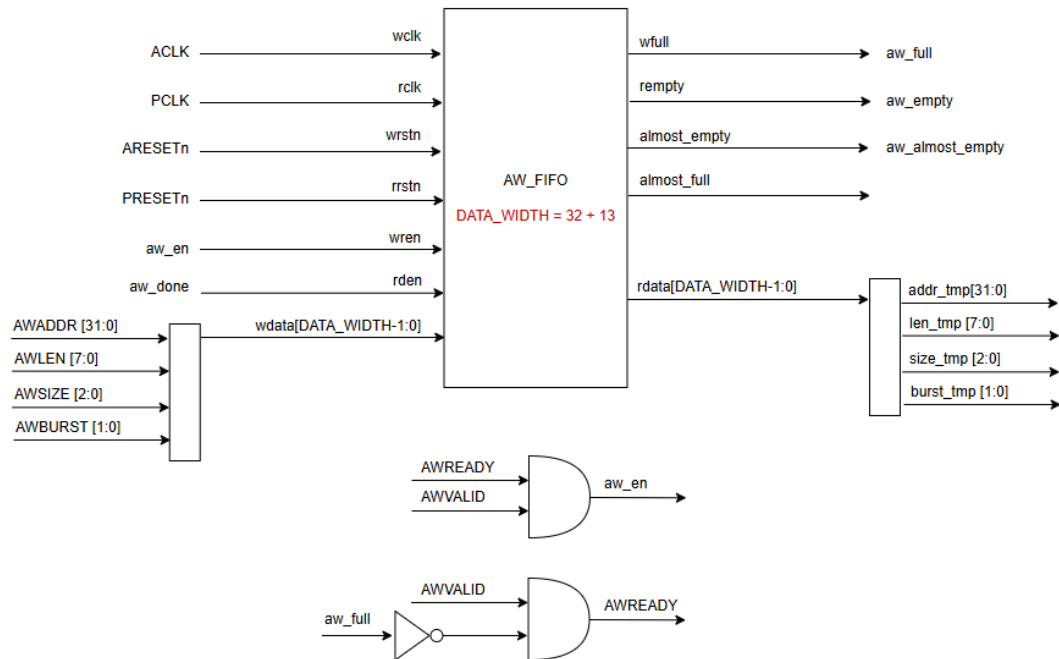
TÍN HIỆU	SỐ BIT	PHÂN LOẠI	MÔ TẢ
wclk	1	Ngõ vào	Tín hiệu xung nhịp dùng cho việc ghi dữ liệu
wrstn	1	Ngõ vào	Tín hiệu reset khởi tạo lại dữ liệu và con trỏ ghi
wren	1	Ngõ vào	Tín hiệu cho phép ghi dữ liệu vào bộ nhớ FIFO
wdata	tùy chỉnh	Ngõ vào	Dữ liệu ghi
rclk	1	Ngõ vào	Tín hiệu xung nhịp dùng cho việc đọc dữ liệu
rrstn	1	Ngõ vào	Tín hiệu reset khởi tạo lại dữ liệu và con trỏ ghi
rden	1	Ngõ vào	Tín hiệu cho phép đọc dữ liệu vào bộ nhớ FIFO
wfull	1	Ngõ ra	Tín hiệu thông báo bộ nhớ FIFO đầy
rdata	tùy chỉnh	Ngõ ra	Dữ liệu đọc
rempty	1	Ngõ ra	Tín hiệu thông báo bộ nhớ FIFO trống
almost_empty	1	Ngõ ra	Tín hiệu thông báo bộ nhớ FIFO gần trống
almost_full	1	Ngõ ra	Tín hiệu thông báo bộ nhớ FIFO gần đầy

Dựa trên bảng mô tả các tín hiệu, có thể thấy khối Asynchronous FIFO được thiết kế để hoạt động độc lập và đồng bộ trên hai miền xung nhịp khác nhau. Cụ thể, các tín hiệu wclk, wrstn, wren và wdata thuộc miền ghi, chịu trách nhiệm tiếp nhận và lưu trữ dữ liệu khi có yêu cầu hợp lệ. Ngược lại, rclk, rrstn, rden và rdata thuộc miền đọc, đảm nhiệm việc xuất dữ liệu theo yêu cầu từ phía nhận. Các tín hiệu trạng thái như wfull, rempty, almost_full và almost_empty đóng vai trò quan trọng trong việc giám sát tình trạng bộ nhớ, ngăn ngừa hiện tượng tràn hoặc đọc dữ liệu rỗng, từ đó đảm bảo tính toàn vẹn và hiệu quả trong quá trình truyền dữ liệu liên miền.

FIFO bất đồng bộ là một khối được thiết kế sẵn, do đó sẽ không được trình bày trong báo cáo. Thay vào đó, sẽ thực hiện phân tích cách kết nối và lưu dữ liệu vào FIFO.

3.2.1.1 Phân tích chi tiết khối AW_FIFO

Khối AW_FIFO dùng để lưu thông tin địa chỉ ghi của beat đầu tiên và các thông tin điều khiển trong một giao dịch ghi. Độ rộng của FIFO được thiết lập để phù hợp để lưu các thông tin của kênh địa chỉ ghi [13].

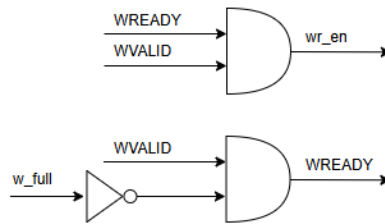
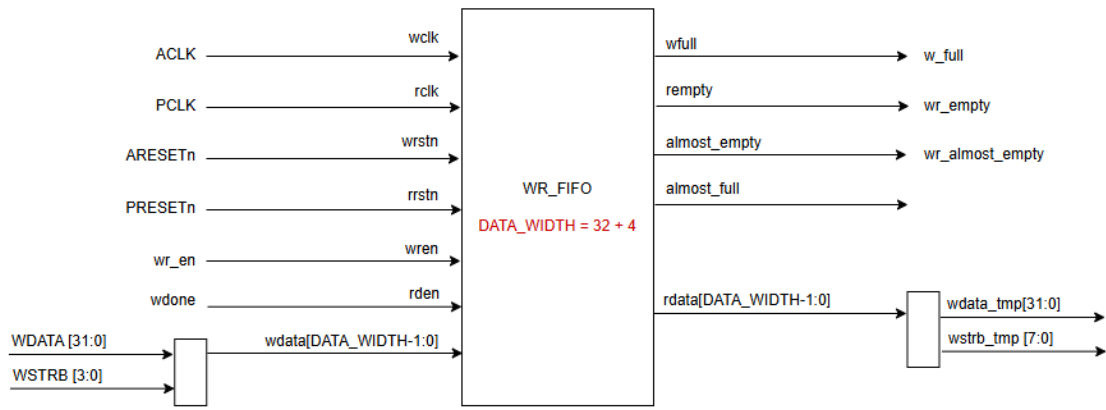


Hình 3.4. Sơ đồ nguyên lý khối AW_FIFO

FIFO cho phép đọc dữ liệu khi tín hiệu `aw_en` là tín hiệu điều khiển việc ghi dữ liệu vào FIFO, và chỉ được ghi dữ liệu vào FIFO khi thỏa mãn các điều kiện: FIFO không đầy và có tín hiệu `AWVALID` báo hiệu việc ghi dữ liệu. Ngoài ra, trước khi ghi dữ liệu vào FIFO, sẽ thực hiện việc ghép các tín hiệu của kênh địa ghi thành một dải dữ liệu, rồi mới đưa vào FIFO. Và khi đọc phải FIFO cũng tiến hành giải dải dữ liệu thành các tín hiệu tương ứng với kênh địa chỉ ghi.

3.2.1.2 Phân tích chi tiết khối WR_FIFO

Khối WR_FIFO dùng để lưu dữ liệu ghi của beat đầu tiên và các thông tin điều khiển trong một giao dịch ghi. Độ rộng của FIFO được thiết lập để phù hợp để lưu các thông tin của kênh dữ liệu ghi [13].

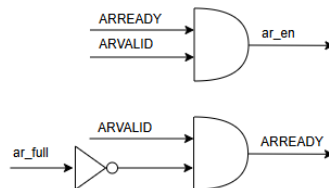
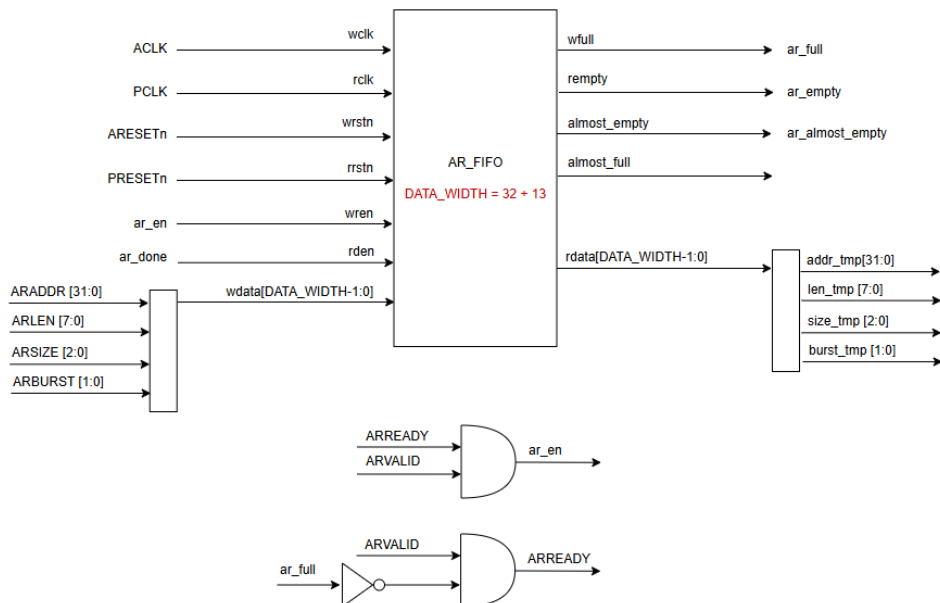


Hình 3.5. Sơ đồ nguyên lý khối *WR_FIFO*

Tương tự việc ghi dữ liệu kênh địa chỉ ghi vào FIFO, việc ghi dữ liệu vào FIFO kênh dữ liệu ghi cũng yêu cầu vào FIFO không đầy và tín hiệu WVALID.

3.2.1.3 Phân tích chi tiết khối *AR_FIFO*

Khối *AR_FIFO* dùng để lưu địa chỉ đọc của beat đầu tiên và các thông tin điều khiển trong một giao dịch đọc. Độ rộng của FIFO được thiết lập để phù hợp để lưu các thông tin của kênh dữ liệu đọc [13].

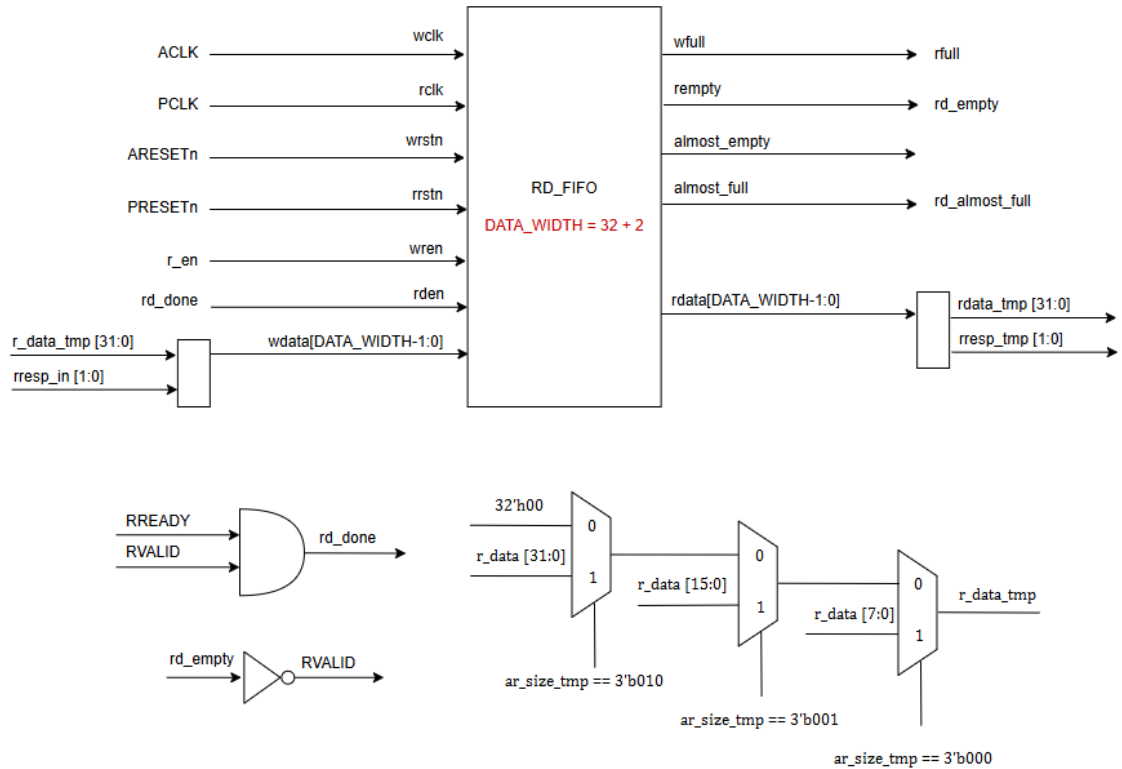


Hình 3.6. Sơ đồ nguyên lý khối *AR_FIFO*

Tương tự việc ghi dữ liệu kênh địa chỉ ghi vào FIFO, việc ghi dữ liệu vào FIFO kênh địa chỉ đọc cũng yêu cầu vào FIFO không đầy và tín hiệu ARVALID.

3.2.1.4 Phân tích chi tiết khối RD_FIFO

Khối RD_FIFO dùng để lưu dữ liệu đọc của beat đầu tiên và các thông tin điều khiển trong một giao dịch đọc. Độ rộng của FIFO được thiết lập để phù hợp để lưu các thông tin của kênh dữ liệu đọc [13].



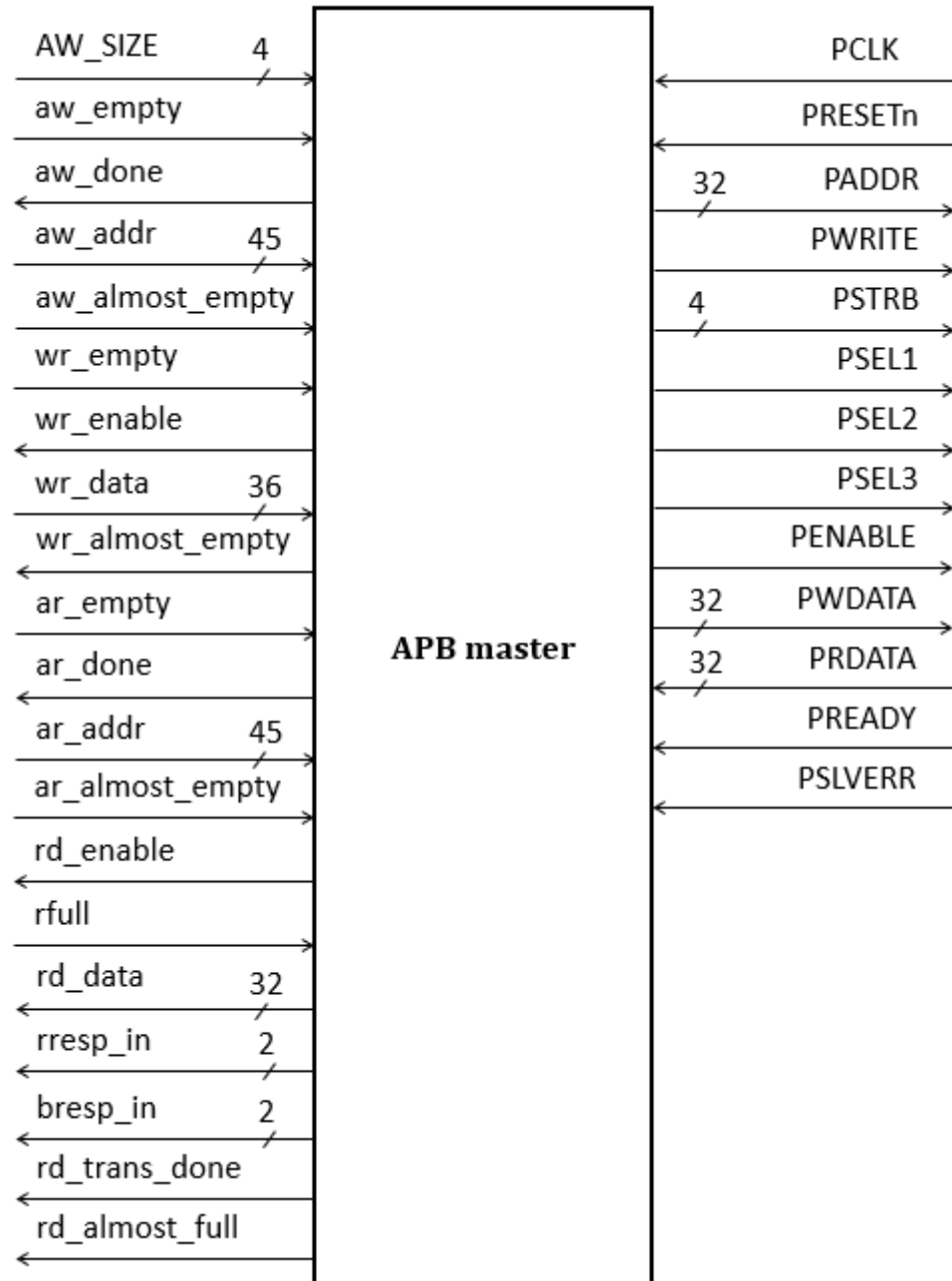
Hình 3.7. Sơ đồ nguyên lý khối RD_FIFO

Dữ liệu ghi vào FIFO phụ thuộc vào tín hiệu ARSIZE cho phép đọc bao nhiêu byte. Bên cạnh đó, tín hiệu RVALID phụ thuộc vào việc FIFO đã có dữ liệu vào FIFO.

3.2.2 Thiết kế khối APB master

Khối APB Master thực hiện chức năng điều khiển các giao dịch đọc và ghi trên bus APB4 dựa trên các yêu cầu từ FIFO đầu vào. Khối bao gồm một FSM quản lý ba trạng thái IDLE, SETUP, ACCESS, kết hợp với khối con RW Arbiter và hai khối giải mã địa chỉ để phân quyền truy cập và sinh địa chỉ cho từng giao dịch. Tại trạng thái SETUP, khối chuẩn bị các tín hiệu điều khiển và dữ liệu, trong khi trạng thái ACCESS thực hiện giao dịch trên bus APB. Khối đồng thời

sinh tín hiệu done, aw_done, ar_done, và dữ liệu đọc (rd_data) để đồng bộ với các phần còn lại của hệ thống. Nhờ cơ chế này, APB Master đảm bảo các giao dịch tuần tự, không xung đột và tuân thủ đặc tả APB4. Hình 3.8. mô tả sơ đồ các chân kết nối của khối APB Master.



Hình 3.8. Sơ đồ tín hiệu kết nối của khối APB master

Bảng 3.3. dưới đây cung cấp một mô tả chi tiết và đầy đủ về chức năng của từng tín hiệu.

Bảng 3.3. Chức năng của các tín hiệu trong khối APB Master

TÍN HIỆU	SỐ BIT	PHÂN LOẠI	MÔ TẢ
PCLK	1	Ngõ vào	Tín hiệu xung nhịp của giao thức APB
PRESETn	1	Ngõ vào	Tín hiệu reset tích cực mức thấp để khởi tạo lại hệ thống
PSTRB	4	Ngõ ra	Tín hiệu cho phép truy cập byte dữ liệu ghi
PADDR	32	Ngõ ra	Tín hiệu địa chỉ chứa thông tin địa chỉ ghi hoặc đọc
PSEL1	1	Ngõ ra	Tín hiệu thông báo Slave 1 được chọn
PSEL2	1	Ngõ ra	Tín hiệu thông báo Slave 2 được chọn
PSEL3	1	Ngõ ra	Tín hiệu thông báo Slave 3 được chọn
PENABLE	1	Ngõ ra	Tín hiệu cho phép đọc hoặc ghi dữ liệu
PWRITE	1	Ngõ ra	Tín hiệu cho phép đọc khi được tích cực
PWDATA	1	Ngõ ra	Dữ liệu ghi
PSTRB	4	Ngõ ra	Tín hiệu cho phép truy cập byte dữ liệu ghi
PREADY	1	Ngõ vào	Tín hiệu cho biết Master sẵn sàng nhận dữ liệu đọc
PRDATA	32	Ngõ vào	Dữ liệu đọc
PSLVERR	1	Ngõ vào	Tín hiệu có lỗi phát sinh trong quá trình đọc hoặc ghi
AW_SIZE	4	Ngõ vào	Tín hiệu kích thước dữ liệu ghi
aw_empty	1	Ngõ vào	Tín hiệu thông báo bộ nhớ FIFO lưu dữ liệu kênh địa chỉ ghi trống
aw_done	1	Ngõ ra	Tín hiệu thông báo hoàn tất giao dịch ghi
aw_addr	45	Ngõ vào	Tín hiệu chứa toàn bộ thông tin của kênh địa chỉ ghi bao gồm: AWADDR, AWLEN, AWSIZE, AWBURST
aw_almost_empty	1	Ngõ vào	Tín hiệu thông báo bộ nhớ FIFO dữ liệu kênh địa chỉ ghi gần trống

ar_empty	1	Ngõ vào	Tín hiệu thông báo bộ nhớ FIFO lưu dữ liệu kênh địa chỉ đọc trống
ar_done	1	Ngõ ra	Tín hiệu thông báo hoàn tất giao dịch đọc
ar_addr	45	Ngõ vào	Tín hiệu chứa toàn bộ thông tin của kênh địa chỉ ghi bao gồm: ARADDR, ARLEN, ARSIZE, ARBURST
ar_almost_empty	1	Ngõ vào	Tín hiệu thông báo bộ nhớ FIFO dữ liệu kênh địa chỉ đọc gần trống
wr_empty	1	Ngõ vào	Tín hiệu thông báo bộ nhớ FIFO dữ liệu kênh dữ liệu ghi trống
wr_enable	1	Ngõ ra	Tín hiệu cho phép khối hoạt động
wr_data	36	Ngõ vào	Tín hiệu chứa thông tin của kênh dữ liệu ghi bao gồm: WDATA và WSTRB
wr_almost_empty	1	Ngõ vào	Tín hiệu thông báo bộ nhớ FIFO dữ liệu kênh dữ liệu ghi gần trống
rd_enable	1	Ngõ ra	Tín hiệu cho phép khối hoạt động
rd_full	1	Ngõ vào	Tín hiệu thông báo bộ nhớ FIFO dữ liệu kênh dữ liệu đọc đầy
rd_data	32	Ngõ ra	Dữ liệu đọc từ APB để đưa vào FIFO đọc.
rresp_in	2	Ngõ ra	Báo lỗi đọc
bresp_in	2	Ngõ ra	Báo lỗi ghi
rd_trans_done	1	Ngõ ra	Tín hiệu báo hoàn tất toàn bộ giao dịch đọc.

Dựa vào bảng 3.3, khối APB Master được thiết kế với một hệ thống tín hiệu đầu vào có vai trò trung tâm trong việc điều khiển quá trình chuyển đổi giao thức từ AXI4 sang APB4. Khối này thực hiện ba chức năng cốt lõi: điều khiển luồng dữ liệu thông qua giám sát trạng thái của các khối FIFO, quản lý việc chuyển đổi và thực thi các giao dịch, điều phối truy cập đến ba slave APB độc lập thông qua cơ chế giải mã địa chỉ và chọn slave tương ứng. Việc lựa chọn slave được thực hiện thông qua tín hiệu PSELx, tín hiệu này sẽ được kích hoạt khi địa chỉ yêu cầu thuộc vùng nhớ được ánh xạ tới slave tương ứng. Các điều

kiện cụ thể cho việc lựa chọn ngoại vi dựa trên giá trị địa chỉ PADDR được thiết kế theo dạng tham số hóa, cho phép người dùng tự do cấu hình địa chỉ ánh xạ đến các slave. Các tham số mặc định được trình bày chi tiết trong Bảng 3.6.

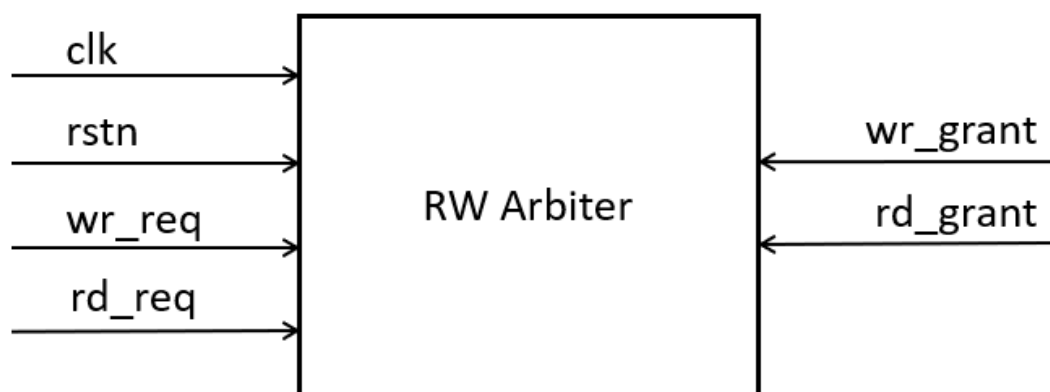
Bảng 3.4. Mô tả điều kiện vùng địa chỉ ánh xạ tới slave tương ứng

TÍN HIỆU PSEL _x TÍCH CỰC	ĐỊA CHỈ BẮT ĐẦU	ĐỊA CHỈ KẾT THÚC
PSEL1	32'h00	32'hFFF
PSEL2	32'h1000	32'h1FFF
PSEL3	32'h2000	32'h2FFF

Khối APB Master được thiết kế dựa trên sự kết hợp của nhiều tín hiệu điều khiển và logic, nhằm tạo ra các chu kỳ giao dịch APB4 tuân thủ nghiêm ngặt đặc tả kỹ thuật. Trọng tâm của khối là máy trạng thái với ba trạng thái chính: IDLE, SETUP và ACCESS. Cơ chế chuyển đổi chi tiết giữa các trạng thái IDLE, SETUP, và ACCESS là một phần tiêu chuẩn và đã được ARM định nghĩa rõ ràng trong tài liệu đặc tả giao thức APB4. Do đó, phần mô tả chi tiết về trình tự chuyển trạng thái và các điều kiện kích hoạt sẽ không được trình bày lại trong chương này. Ngoài ra, trong khối có kết hợp hai khối con là RW Arbiter và khối giải mã địa chỉ, đảm nhận chức năng phân quyền đọc ghi và giải mã địa chỉ theo cơ chế truyền dữ liệu.

3.2.2.1 Thiết kế khối RW Arbiter

Khối RW Arbiter chịu trách nhiệm điều phối và phân quyền truy cập giữa hai yêu cầu đọc và ghi trên bus APB. Dựa trên trạng thái hiện tại của bộ điều khiển, arbiter tạo ra hai tín hiệu cấp quyền wr_grant và rd_grant, trong đó mỗi thời điểm chỉ cho phép một loại giao dịch được thực thi. Cơ chế này giúp đảm bảo quá trình truy cập diễn ra tuần tự, loại bỏ xung đột giữa các yêu cầu đồng thời, và duy trì việc điều khiển bus APB luôn nằm trong tác vụ hợp lệ [14]. Hình 3.9. mô tả sơ đồ các ngõ vào và ngõ ra của khối RW Arbiter.



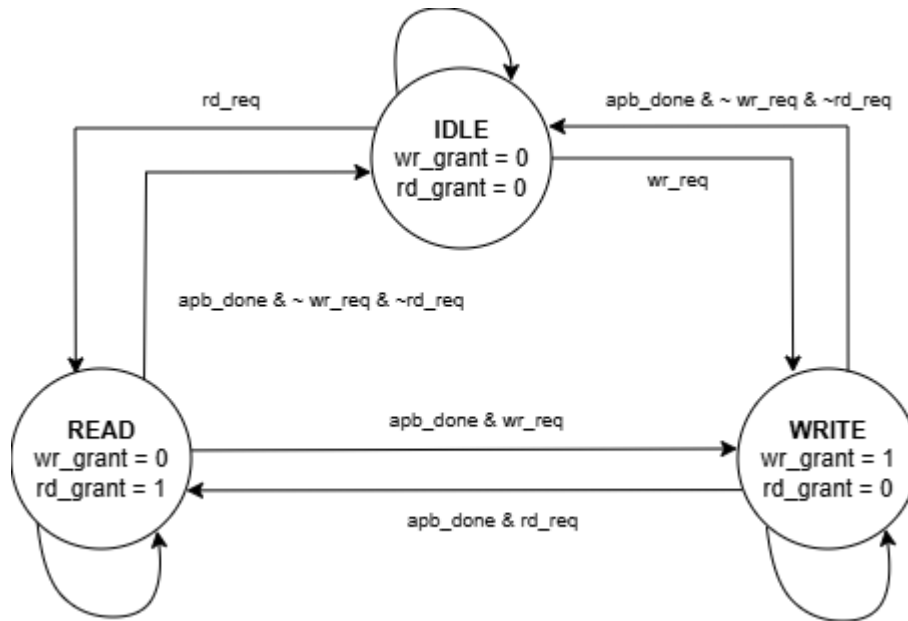
Hình 3.9. Sơ đồ tín hiệu kết nối của khối RW Arbiter

Bảng 3.5. dưới đây cung cấp một mô tả chi tiết và đầy đủ về chức năng của từng tín hiệu.

Bảng 3.5. Chức năng của các tín hiệu trong khối RW Arbiter

TÍN HIỆU	SỐ BIT	PHÂN LOẠI	MÔ TẢ
clk	1	Ngõ vào	Tín hiệu xung nhịp đồng bộ hệ thống.
rstn	1	Ngõ vào	Tín hiệu reset tích cực mức thấp để khởi tạo lại hệ thống.
wr_req	1	Ngõ vào	Tín hiệu yêu cầu thực hiện giao dịch ghi.
rd_req	1	Ngõ vào	Tín hiệu yêu cầu thực hiện giao dịch đọc.
apb_done	1	Ngõ vào	Tín hiệu cho biết giao dịch hiện tại hoàn tất.
wr_grant	1	Ngõ ra	Tín hiệu cấp quyền cho phép thực hiện giao dịch ghi.
rd_grant	1	Ngõ ra	Tín hiệu cấp quyền cho phép thực hiện giao dịch đọc.

Hai tín hiệu clk và rstn lần lượt cung cấp xung nhịp đồng bộ và điều kiện khởi tạo lại cho toàn bộ khối RW Arbiter. Hai tín hiệu wr_req và rd_req đại diện cho yêu cầu ghi và yêu cầu đọc từ phía AXI, là cơ sở để arbiter quyết định phân quyền truy cập. Tín hiệu apb_done được sử dụng để thông báo giao dịch APB hiện tại đã hoàn tất, cho phép arbiter chuyển sang cấp quyền cho yêu cầu tiếp theo. Đầu ra wr_grant và rd_grant lần lượt cấp quyền thực hiện giao dịch ghi hoặc đọc, đảm bảo mỗi thời điểm chỉ một loại tác vụ được phép truy cập bus APB. Hoạt động của khối RW Arbiter được mô tả trong hình 3.10. dưới đây.



Hình 3.10. Máy trạng thái của khối RW Arbiter

Mô tả chi tiết hình 3.5, máy trạng thái bao gồm ba trạng thái hoạt động chính, mỗi trạng thái đảm nhận một chức năng cụ thể nhằm đảm bảo quá trình phân quyền diễn ra tuần tự, tránh xung đột và duy trì tính ổn định của hệ thống. Chức năng của từng trạng thái được mô tả như sau:

IDLE: Đây là trạng thái khởi tạo và trạng thái nghỉ của hệ thống, nơi arbiter chưa cấp quyền cho bất kỳ giao dịch nào. Trong trạng thái này, arbiter liên tục theo dõi các yêu cầu đọc và ghi để quyết định chuyển sang chế độ xử lý phù hợp.

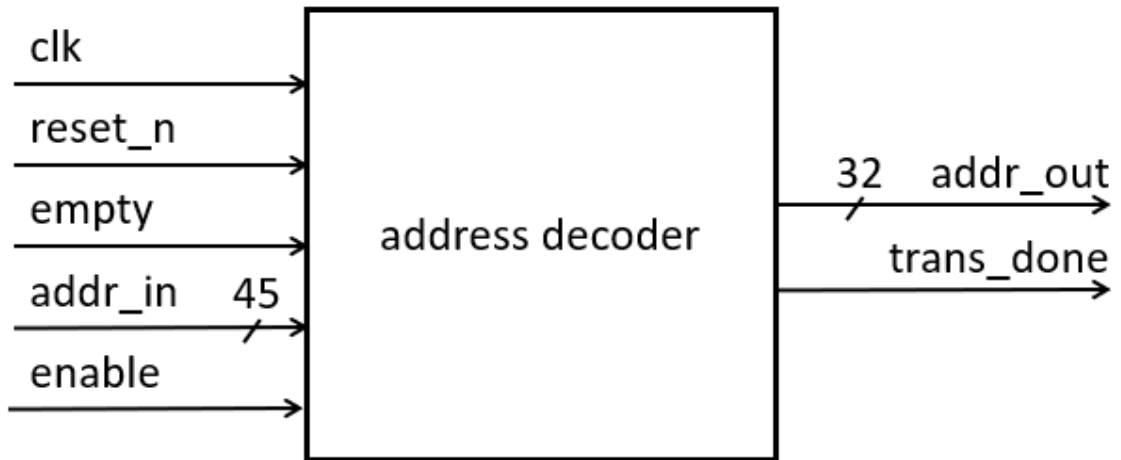
READ: thể hiện arbiter đang cấp quyền cho một giao dịch đọc trên bus APB. Trạng thái được duy trì cho đến khi tín hiệu `apb_done` xuất hiện, cho biết giao dịch đã hoàn tất. Sau đó, arbiter đánh giá lại các yêu cầu đang chờ để xác định trạng thái kế tiếp.

WRITE: được kích hoạt khi arbiter cấp quyền cho một giao dịch ghi. Tương tự trạng thái READ, arbiter duy trì trạng thái này cho đến khi nhận được tín hiệu `apb_done`, trước khi chuyển sang trạng thái phù hợp dựa trên các yêu cầu còn lại.

3.2.2.2 Thiết kế khối giải mã địa chỉ

Khối giải mã địa chỉ thực hiện giải mã và phát sinh địa chỉ truy cập cho từng chu kỳ truyền dữ liệu trong giao thức AXI4, dựa trên địa chỉ gốc, độ dài burst, kích thước dữ liệu và loại burst (FIXED, INCR, WRAP). Khối đồng thời theo dõi bộ đếm nhịp truyền để xác định thời điểm hoàn tất giao dịch và chuẩn bị

cho giao dịch tiếp theo, đảm bảo địa chỉ phát sinh chính xác và tuân thủ đặc tả AXI4. Hình 3.7. mô tả sơ đồ các ngõ vào và ngõ ra của khối RW Arbiter.



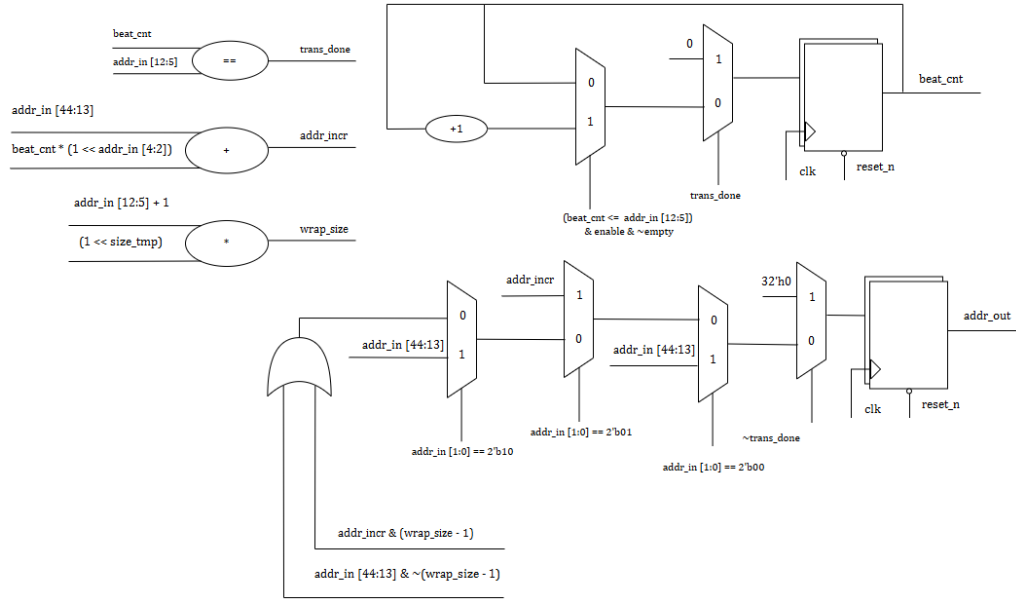
Hình 3.11. Sơ đồ tín hiệu kết nối của khối giải mã địa chỉ

Bảng 3.5. dưới đây cung cấp một mô tả chi tiết và đầy đủ về chức năng của từng tín hiệu.

Bảng 3.5. Chức năng của các tín hiệu trong khối giải mã địa chỉ

TÍN HIỆU	SỐ BIT	PHÂN LOẠI	MÔ TẢ
clk	1	Ngõ vào	Tín hiệu xung nhịp đồng bộ hệ thống.
reset_n	1	Ngõ vào	Tín hiệu reset tích cực mức thấp để khởi tạo lại hệ thống.
empty	1	Ngõ vào	Tín hiệu thông báo bộ nhớ FIFO trống
addr_in	45	Ngõ vào	Dữ liệu đầu vào chứa địa chỉ gốc, độ dài burst, kích thước dữ liệu và loại burst
enable	1	Ngõ vào	Tín hiệu cho phép khối hoạt động
addr_out	45	Ngõ ra	Dữ liệu đầu ra chứa địa chỉ gốc, độ dài burst, kích thước dữ liệu và loại burst
trans_done	1	Ngõ ra	Tín hiệu báo hiệu hoàn tất một giao dịch burst

Dựa theo bảng 3.5, một số tín hiệu ngõ vào có vai trò điều khiển hoạt động của khối giải mã địa chỉ. Hình 3.8. minh họa các kiến trúc được sử dụng trong việc xây dựng khối.



Hình 3.12. Sơ đồ kiến trúc của khối giải mã địa chỉ

Khối giải mã địa chỉ hoạt động dựa trên cơ chế đồng bộ sử dụng Flip Flop D để lưu trữ giá trị địa chỉ và theo dõi tiến trình truyền dữ liệu thông qua bộ đếm beat (beat_cnt). Bộ đếm này liên tục so sánh với độ dài burst để xác định thời điểm kết thúc giao dịch, được báo hiệu bằng tín hiệu trans_done.

Đồng thời, khối tính toán địa chỉ xử lý song song cho ba chế độ burst: Với INCR, địa chỉ được tính bằng cách cộng dồn địa chỉ cơ sở với tích số beat_cnt và kích thước truyền. Đối với WRAP, hệ thống xác định vùng bao qua wrap_size, tính toán wrap_base và wrap_offset, rồi kết hợp thông qua phép OR. Các kết quả này được chọn lựa bởi bộ multiplexer dựa trên loại burst.

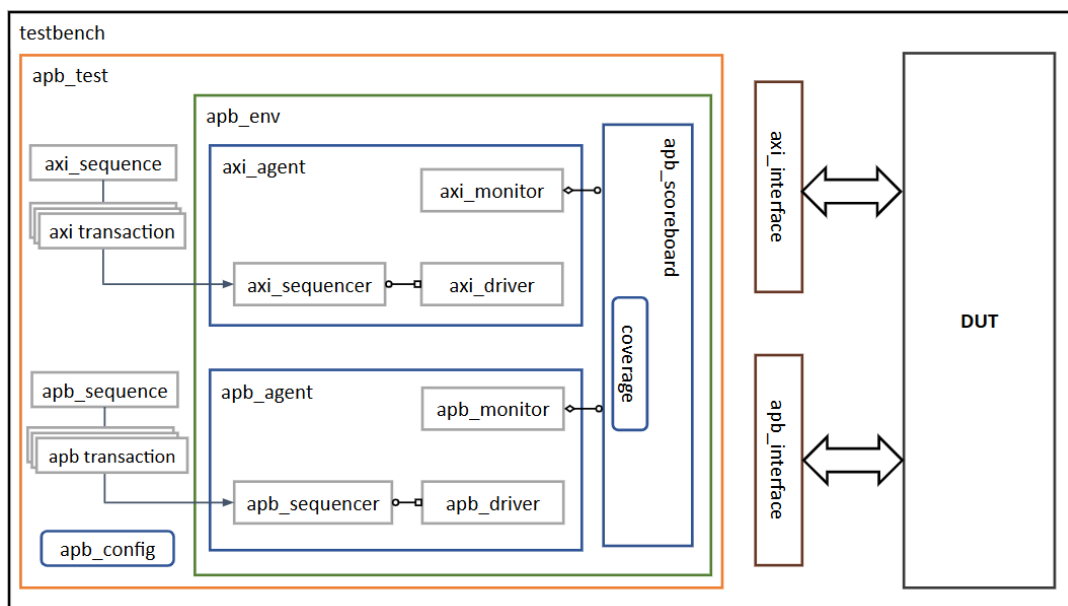
3.3 KIẾN TRÚC MÔI TRƯỜNG KIỂM THỬ UVM

Testbench được xây dựng dựa trên phương pháp xác minh nhằm kiểm tra chức năng bộ chuyển đổi giao thức AXI4 sang APB4. Cấu trúc testbench được tổ chức theo mô hình phân cấp, mỗi thành phần đảm nhiệm một chức năng riêng biệt. Các thành phần được liệt kê như sau:

- testbench: mức cao nhất của UVM testbench.
- apb_test: là lớp cơ sở cho các kịch bản kiểm thử, trong đó chứa môi trường kiểm thử.
- apb_env: môi trường xác minh, bao gồm agent, scoreboard và giao diện kết nối với DUT.

- axi_agent, apb_agent: bao gồm sequencer, driver và monitor.
- axi_interface, apb_interface: điểm kết nối giữa các thành phần của testbench và DUT. Trong các interface này sử dụng chức năng clocking block để đảm bảo tín hiệu được lấy mẫu và điều khiển đồng bộ theo xung nhịp, hạn chế sai lệch thời gian trong quá trình mô phỏng. Ngoài ra, interface còn tích hợp thêm SystemVerilog Assertions (SVA) để ràng buộc các thời gian quan trọng của hai giao thức, giúp phát hiện sớm các vi phạm đặc tả giao thức trong quá trình xác minh.
- DUT: khối phần cứng cần kiểm thử.
- apb_scoreboard: thu thập và so sánh kết quả đầu ra thực tế của DUT với kết quả mong đợi.
- Ngoài ra, trong môi trường có sử dụng Transaction-Level Modeling (TLM) để kết nối và truyền dữ liệu giữa các thành phần, từ sequencer sang driver, từ monitor sang scoreboard. Đồng thời, cơ chế uvm_config_db được sử dụng để truyền các tham số hoặc virtual interface xuống các thành phần con, đảm bảo tính nhất quán, tăng khả năng tái sử dụng của testbench.

Hình ảnh minh họa xây dựng môi trường thể hiện ở hình 3.13. dưới đây.



Hình 3.13. Kiến trúc môi trường kiểm thử UVM cho bộ chuyển đổi

Cấu trúc phân cấp của toàn bộ testbench sau khi chạy tệp kiểm tra được thể hiện ở hình 3.14. dưới đây. Ngoài các thành phần vừa được liệt kê ở trên, còn có các thành phần như arbitration_queue và lock_queue được UVM sử dụng để quản lý việc phân xử và đồng bộ các giao dịch.

#	Name	Type	Size	Value
#	uvm_test_top	wr_incr_psel2_4byte_error_test	-	@351
#	apb_env	apb_environment	-	@369
#	apb_agt	apb_agent	-	@432
#	driver	apb_driver	-	@462
#	rsp_port	uvm_analysis_port	-	@481
#	seq_item_port	uvm_seq_item_pull_port	-	@471
#	monitor	apb_monitor	-	@633
#	apb_item_act	uvm_analysis_port	-	@642
#	sequencer	apb_sequencer	-	@491
#	rsp_export	uvm_analysis_export	-	@500
#	seq_item_export	uvm_seq_item_pull_imp	-	@618
#	arbitration_queue	array	0	-
#	lock_queue	array	0	-
#	num_last_reqs	integral	32	'd1
#	num_last_rsps	integral	32	'd1
#	apb_sb	apb_scoreboard	-	@394
#	APB_IP	apb_coverage	-	@403
#	apb_imp	uvm_analysis_imp_apb	-	@422
#	axi_imp	uvm_analysis_imp_axi	-	@412
#	apb_item_exp	uvm_analysis_imp_apb	-	@675
#	axi_item_exp	uvm_analysis_imp_axi	-	@665
#	axi_agt	axi_agent	-	@441
#	driver	axi_driver	-	@690
#	rsp_port	uvm_analysis_port	-	@709
#	seq_item_port	uvm_seq_item_pull_port	-	@699
#	monitor	axi_monitor	-	@859
#	axi_item_act	uvm_analysis_port	-	@868
#	sequencer	axi_sequencer	-	@719
#	rsp_export	uvm_analysis_export	-	@728
#	seq_item_export	uvm_seq_item_pull_imp	-	@846
#	arbitration_queue	array	0	-
#	lock_queue	array	0	-
#	num_last_reqs	integral	32	'd1
#	num_last_rsps	integral	32	'd1

Hình 3.14. Cấu trúc phân cấp sau thiết kế của UVM

CHƯƠNG 4

KẾT QUẢ THỰC HIỆN

4.1 TÓM TẮT NỘI DUNG

Trong chương này, chúng tôi tập trung vào việc kiểm tra hoạt động của bộ chuyển đổi giao thức sử dụng môi trường kiểm thử UVM đã thiết kế ở chương 3. Bộ chuyển đổi giao thức được thiết kế tuân theo đặc tả của hãng AMBA, trong khi môi trường UVM được xây dựng nhằm tạo ra các kịch bản truyền dữ liệu đa dạng trên giao thức AXI4, bao gồm các loại truyền dữ liệu, độ dài truyền và trường hợp có lỗi phát sinh. Scoreboard được sử dụng để đối chiếu giữa giao dịch AXI4 và giao dịch APB4, từ đó kiểm chứng mức độ chính xác của thiết kế.

Bên cạnh kiểm thử chức năng, đồ án thực hiện thu thập thông số tài nguyên sử dụng, công suất tiêu thụ, định thời (timing) trên nền tảng phần cứng ở nhiều mức tần số xung nhịp để đánh giá chi phí tài nguyên và khả năng hoạt động thực tế của bộ chuyển đổi. Đồng thời, các chỉ số độ bao phủ và độ bao phủ chức năng được phân tích để đánh giá mức độ đầy đủ của quá trình kiểm thử chức năng của thiết kế. Từ đó, đưa ra nhận xét về tính khả thi khi tích hợp vào hệ thống SoC.

4.2 MÔ PHỎNG HỆ THỐNG

Trong phần này, chúng tôi tiến hành xây dựng hơn 90 tệp kiểm tra của quá trình đọc và quá trình ghi với các thông số đầu vào cho từng trường hợp tệp kiểm tra khác nhau nhằm mục đích bao quát hết tất cả chức năng của thiết kế. Tuy nhiên, trong báo cáo này, chúng tôi chỉ trình bày 9 tệp kiểm tra đại diện chức năng chính của thiết kế, các tệp kiểm tra còn lại sẽ được đính kèm ở đường truy cập đính kèm theo. Điểm chung duy nhất của toàn bộ tệp kiểm tra là xung nhịp của giao thức AXI4 và APB4 được thiết lập với tần số lần lượt là 200MHz và

100MHz. Các tham số cấu hình địa chỉ lựa chọn khối APB4 con được giữ nguyên thiết lập như lúc thiết kế.

Bảng 4.1 dưới đây mô tả tất cả các tập kiểm tra của quá trình đọc và quá trình ghi với nhiều trường hợp dữ liệu, độ dài dữ liệu và số lượng beat dữ liệu truyền khác nhau đối với từng trường hợp cơ chế truyền khác nhau.

Bảng 4.1. Giới thiệu các testcase

TESTCASE	NỘI DUNG KIỂM TRA
Quá trình ghi	
1	Mục tiêu: Kiểm tra việc truyền dữ liệu ngoại vi 1 theo cơ chế FIXED. Thông số đầu vào: - 1 đơn vị truyền dữ liệu - Độ dài dữ liệu là 1 byte cho mỗi lần truyền dữ liệu - Không cấu hình lỗi
2	Mục tiêu: Kiểm tra truyền dữ liệu ngoại vi 1, 2 theo cơ chế INCR. Thông số đầu vào: - 4 đơn vị truyền dữ liệu - Độ dài dữ liệu là 4 bytes cho mỗi lần truyền dữ liệu - Không cấu hình lỗi
3	Mục tiêu: Kiểm tra việc truyền dữ liệu ngoại vi 2, 3 theo cơ chế WRAP. Thông số đầu vào: - 8 đơn vị truyền dữ liệu - Độ dài dữ liệu là 4 bytes cho mỗi lần truyền dữ liệu - Không cấu hình lỗi
4	Mục tiêu: Kiểm tra việc phản hồi lỗi trong quá trình ghi. Thông số đầu vào: - Truyền dữ liệu ngoại vi 3 theo kiểu FIXED. - 1 đơn vị truyền dữ liệu - Độ dài dữ liệu là 4 bytes cho mỗi lần truyền dữ liệu

Quá trình đọc	
5	Mục tiêu: Kiểm tra truyền dữ liệu ngoại vi 3 theo cơ chế FIXED. Thông số đầu vào: - 1 đơn vị truyền dữ liệu - Độ dài dữ liệu là 2 bytes cho mỗi lần truyền dữ liệu - Không cấu hình lỗi
6	Mục tiêu: Kiểm tra truyền dữ liệu ngoại vi 2, 3 theo cơ chế INCR. Thông số đầu vào: - 8 đơn vị truyền dữ liệu - Độ dài dữ liệu là 4 bytes cho mỗi lần truyền dữ liệu - Không cấu hình lỗi
7	Mục tiêu: Kiểm tra truyền dữ liệu ngoại vi 1, 2 theo cơ chế WRAP. Thông số đầu vào: - 8 đơn vị truyền dữ liệu. Không cấu hình lỗi - Độ dài dữ liệu là 4 bytes cho mỗi lần truyền dữ liệu
8	Mục tiêu: Kiểm tra việc phản hồi lỗi trong quá trình đọc. Thông số đầu vào: - Truyền dữ liệu ngoại vi 2 theo kiểu FIXED. - 1 đơn vị truyền dữ liệu - Độ dài dữ liệu là 4 bytes cho mỗi lần truyền dữ liệu
Quá trình ghi và đọc	
9	Mục tiêu: Kiểm tra việc truyền dữ liệu ngoại vi 1, 2 và 3 theo cơ chế WRAP. Thông số đầu vào: - 8 đơn vị truyền dữ liệu - Độ dài dữ liệu là 4 bytes cho mỗi lần truyền dữ liệu - Không cấu hình lỗi

4.2.1 Hoạt động ghi dữ liệu

Xây dựng các tệp kiểm tra của quá trình ghi với mục tiêu kiểm thử các trường hợp để mô phỏng quá trình truyền dữ liệu bao gồm các trường hợp:

- Các cơ chế truyền burst: FIXED, INCR, WRAP.
- Kích thước dữ liệu và độ dài truyền khác nhau.
- Phát sinh lỗi trong quá trình ghi hoặc đọc.
- Truyền nhiều giao dịch.

Điểm chung của các tệp kiểm tra này là tại thời điểm 25ns đến 45ns, là thời gian kênh địa chỉ ghi bắt đầu gửi dữ liệu của kênh địa chỉ ghi. Thời gian tiếp theo là thời gian các kênh dữ liệu ghi, kênh phản hồi và các tín hiệu của giao thức APB4 ghi hoạt động cho đến khi kết thúc một giao dịch ghi.

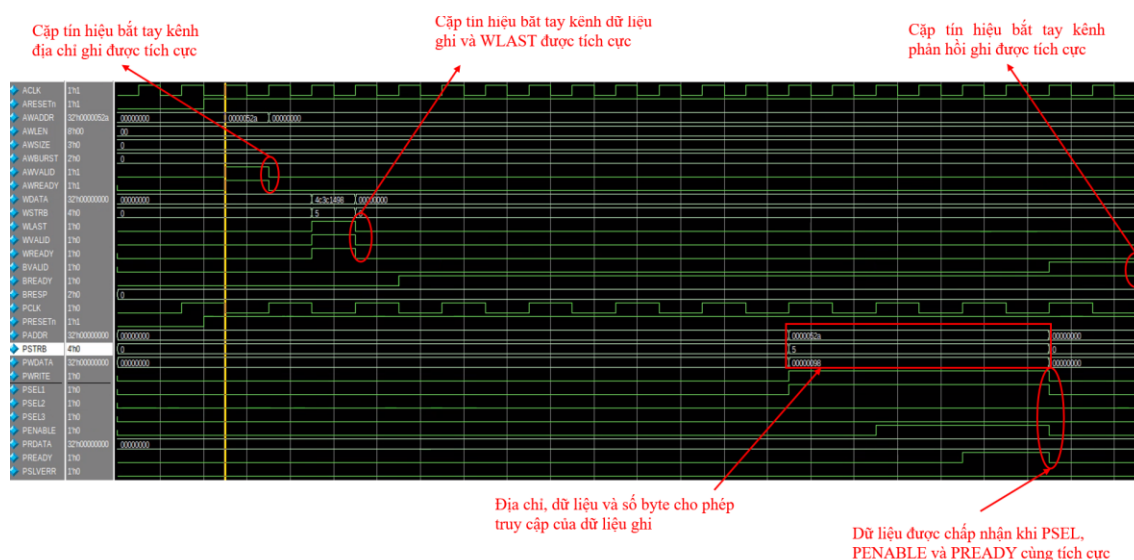
4.2.1.1 Tệp kiểm tra 1

Trong tệp kiểm tra 1, chúng tôi tiến hành kiểm tra quá trình ghi dữ liệu theo cơ chế FIXED. Các thông số đầu vào được ghi ở bảng 4.2 dưới đây.

Bảng 4.2. Thông số đầu vào của tệp kiểm tra 1

TÍN HIỆU ĐẦU VÀO	GIÁ TRỊ
AWADDR	Ngẫu nhiên từ 32'h00 đến 32'hFFF
AWLEN	8'd00
AWSIZE	3'b000
AWBURST	2'b00

Ta quan sát kết quả mô phỏng trong hình 4.1 dưới đây.



Hình 4.1. Dạng sóng tệp kiểm tra 1 của quá trình ghi

Tại 25ns, tín hiệu AWVALID và AWREADY cùng tích cực, cơ chế bắt tay của kênh địa chỉ ghi được thực hiện, thông tin địa chỉ ghi được chấp nhận. Tại 45ns, tín hiệu WVALID và WREADY cùng tích cực, cơ chế bắt tay của kênh dữ liệu ghi được hình thành, dữ liệu ghi được chấp nhận. Đối với trường hợp này, số beat truyền đi là 1, do đó sau khi gửi dữ liệu đi một chu kỳ xung nhịp thì đã hoàn thành việc gửi dữ liệu đi một beat. Tín hiệu WLAST được tích cực, xác nhận việc gửi dữ liệu hoàn tất. Tại thời điểm 155ns, quá trình chuyển đổi dữ liệu từ giao thức AXI4 sang giao thức APB4 bắt đầu. Lúc này, địa chỉ chuyển đổi là 32'h52A, tín hiệu PSTRB là 4'b0101, dữ liệu ghi của giao thức AXI4 là 32'h4c3c1498, số byte truyền đi là 1 byte, do đó, tín hiệu PWDATA có giá trị là 32'h98. Điều này cho thấy, việc chuyển đổi giao thức đang thực hiện đúng với đặc tả của các giao thức. Tại 215ns, giao thức APB4 hoàn thành xong một giao dịch ghi dữ liệu. Tín hiệu BRESP trả về giá trị 2'b00. Quá trình chuyển đổi dữ liệu ghi hoàn tất và không phát sinh lỗi.

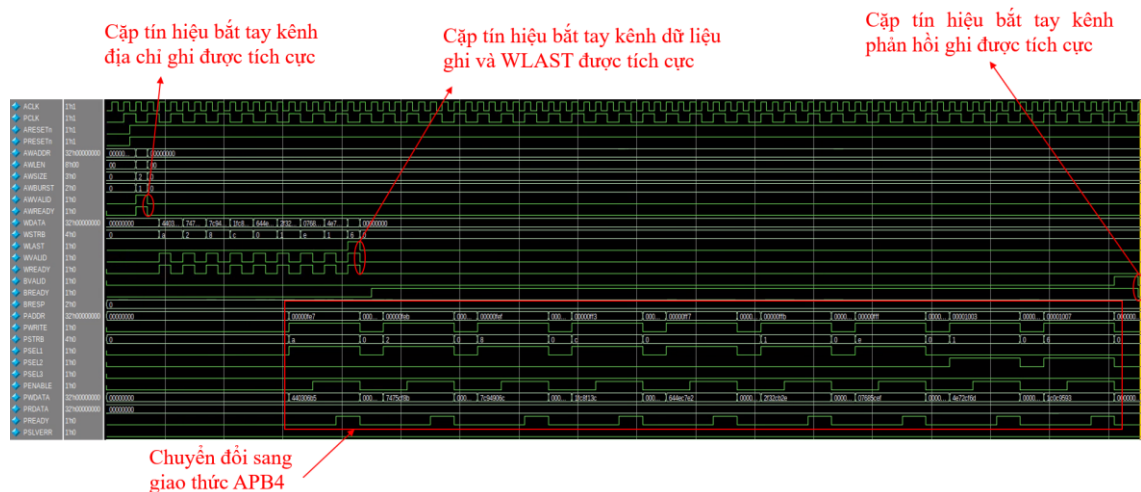
4.2.1.2 Tập kiểm tra 2

Trong tập kiểm tra 2, chúng tôi tiến hành kiểm tra quá trình ghi dữ liệu theo cơ chế INCR. Bắt đầu quá trình ghi theo cơ chế INCR, truyền 8 beat và 4 bytes cho một giao dịch ghi. Các thông số đầu vào được ghi ở bảng 4.3 dưới đây.

Bảng 4.3. Thông số đầu vào của tập kiểm tra 2

TÍN HIỆU ĐẦU VÀO	GIÁ TRỊ
AWADDR	Ngẫu nhiên từ 32'hFE7 đến 32'hFF0
AWLEN	8'd7
AWSIZE	3'b10
AWBURST	2'b01

Ta quan sát kết quả mô phỏng trong hình 4.2 dưới đây.



Hình 4.2. Dạng sóng tệp kiểm tra 2 của quá trình ghi

Bộ chuyển đổi thực hiện tương tự các quy tắc như ở tệp kiểm tra 1, tuy nhiên, địa chỉ ghi sẽ tuân theo cơ chế truyền INCR. Tại thời điểm 25ns, tín hiệu AWVALID và AWREADY cùng tích cực, lúc này kênh địa chỉ ghi hoàn tất việc truyền dữ liệu. Tại 45ns, dữ liệu của kênh dữ liệu bắt đầu truyền dữ liệu. Tại 155ns, các tín hiệu của giao thức APB bắt đầu truyền dữ liệu qua ngoại vi. Từ 155ns đến 215ns, địa chỉ ghi dữ liệu tăng dần theo đơn vị kích thước dữ liệu. Tại 215ns, kênh dữ liệu ghi hoàn tất việc truyền dữ liệu. Thời điểm 855ns, hoàn thành việc chuyển đổi dữ liệu sang APB4. Tại 875ns, kênh phản hồi ghi hoàn tất việc phản hồi ghi. Quá trình chuyển đổi giao thức hoàn tất và không phát sinh lỗi trong quá trình chuyển đổi.

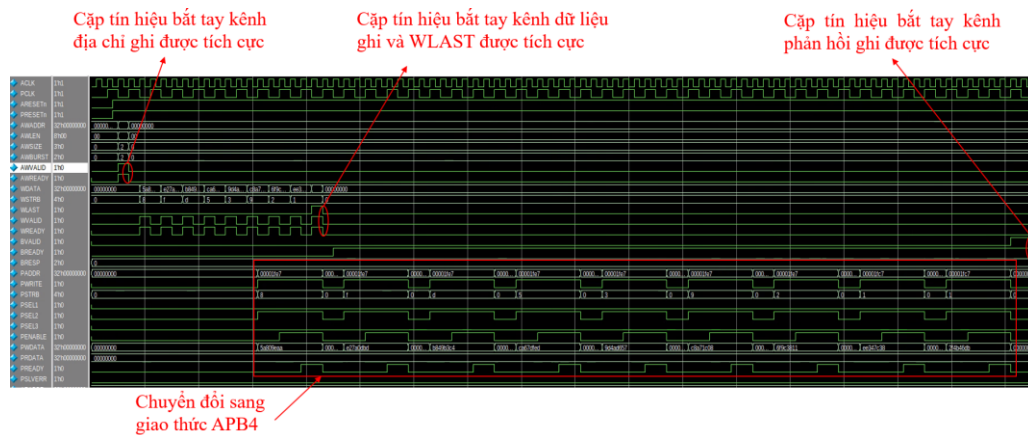
4.2.1.3 Tệp kiểm tra 3

Trong tệp kiểm tra 3, chúng tôi tiến hành kiểm tra quá trình ghi dữ liệu theo cơ chế WRAP. Bắt đầu quá trình ghi theo cơ chế WRAP, truyền 8 beat và 4 bytes cho một giao dịch ghi. Các thông số đầu vào được ghi ở bảng 4.3 dưới đây.

Bảng 4.3. Thông số đầu vào của tệp kiểm tra 3

TÍN HIỆU ĐẦU VÀO	GIÁ TRỊ
AWADDR	Ngẫu nhiên từ 32'h1FE7 đến 32'h1FF0
AWLEN	8'd7
AWSIZE	3'b10
AWBURST	2'b10

Ta quan sát kết quả mô phỏng trong hình 4.3 dưới đây.



Hình 4.3. Dạng sóng tập kiểm tra 3 của quá trình ghi

Bộ chuyển đổi thực hiện tương tự các quy tắc như ở tập kiểm tra 1, tuy nhiên, địa chỉ ghi sẽ tuân theo cơ chế truyền WRAP. Tại thời điểm 25ns, tín hiệu AWVALID và AWREADY cùng tích cực, lúc này kênh địa chỉ ghi hoàn tất việc truyền dữ liệu. Tại 35ns, dữ liệu của kênh dữ liệu bắt đầu truyền dữ liệu. Tại thời điểm 155ns, các tín hiệu của giao thức APB4 bắt đầu truyền dữ liệu qua ngoại vi. Tại 215ns, kênh dữ liệu ghi hoàn tất truyền dữ liệu. Thời điểm 855ns, bộ chuyển đổi giao thức hoàn thành việc chuyển đổi dữ liệu sang APB4. Tại 875ns, kênh phản hồi ghi hoàn tất việc phản hồi ghi. Quá trình chuyển đổi giao thức hoàn tất và không phát sinh lỗi trong quá trình chuyển đổi.

4.2.1.4 Tập kiểm tra 4

Trong tập kiểm tra 4, chúng tôi tiến hành kiểm tra quá trình ghi dữ liệu và trong quá trình ghi dữ liệu, giao thức APB4 phản hồi có lỗi phát sinh. Các thông số đầu vào được ghi ở bảng 4.4 dưới đây.

Bảng 4.4. Thông số đầu vào của tập kiểm tra 4

TÍN HIỆU ĐẦU VÀO	GIÁ TRỊ
AWADDR	Ngẫu nhiên từ 32'h1000 đến 32'h1FFF
AWLEN	8'd0
AWSIZE	3'b10
AWBURST	2'b00
PSLVERR	1

theo là thời gian các kênh dữ liệu đọc, các tín hiệu của giao thức APB4 ghi hoạt động cho đến khi kết thúc một giao dịch đọc.

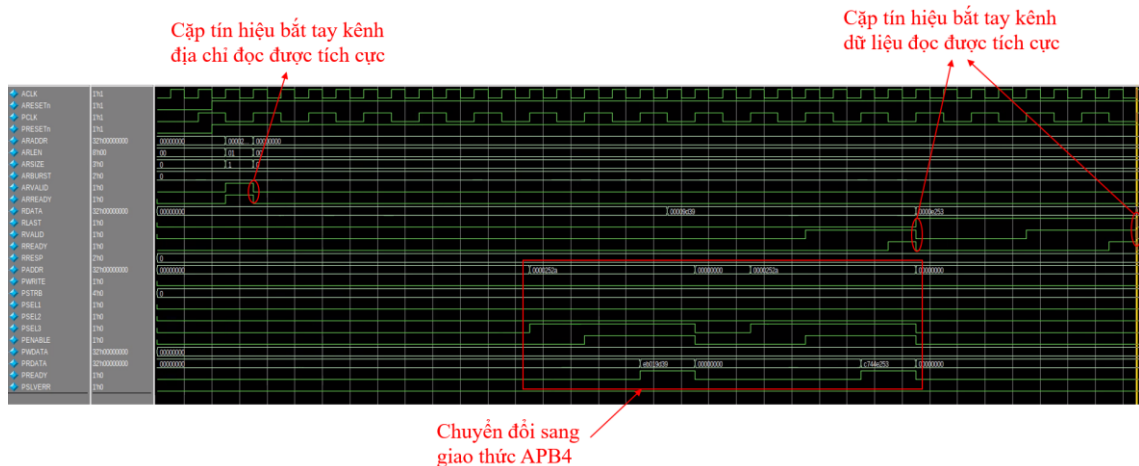
4.2.2.1 Tập kiểm tra 5

Trong tập kiểm tra 5, chúng tôi tiến hành kiểm tra quá trình đọc dữ liệu theo cơ chế FIXED. Bắt đầu quá trình ghi theo cơ chế FIXED, truyền 4 beat và 1 bytes cho một giao dịch ghi. Các thông số đầu vào được ghi ở bảng 4.6 dưới đây.

Bảng 4.6. Thông số đầu vào của tập kiểm tra 5

TÍN HIỆU ĐẦU VÀO	GIÁ TRỊ
ARADDR	Ngẫu nhiên từ 32'h2000 đến 32'h2FFF
ARLEN	8'd1
ARSIZE	3'b01
ARBURST	2'b00

Ta quan sát kết quả mô phỏng trong hình 4.5. dưới đây.



Hình 4.5. Dạng sóng tập kiểm tra 5 của quá trình đọc

Tại thời điểm 25ns, tín hiệu ARVALID và ARREADY cùng tích cực, lúc này kênh địa chỉ đọc hoàn tất việc truyền dữ liệu. Tại 135ns, các tín hiệu của giao thức APB bắt đầu đọc dữ liệu từ ngoại vi. Thời điểm 185ns, bộ chuyển đổi thực hiện việc chuyển đổi beat đầu tiên từ APB4 sang AXI4. Tại 275ns là thời điểm ngoại vi bên APB4 hoàn thành việc truyền dữ liệu. Đồng thời tại thời điểm này, tín hiệu RLAST tích cực, biểu thị transfer cuối cùng trong giao dịch đã được nhận. Tại 355ns, kênh dữ liệu đọc hoàn tất việc đọc dữ liệu. Toàn bộ quá trình chuyển đổi giao thức tuân thủ đúng đặc tả AMBA và không phát sinh lỗi.

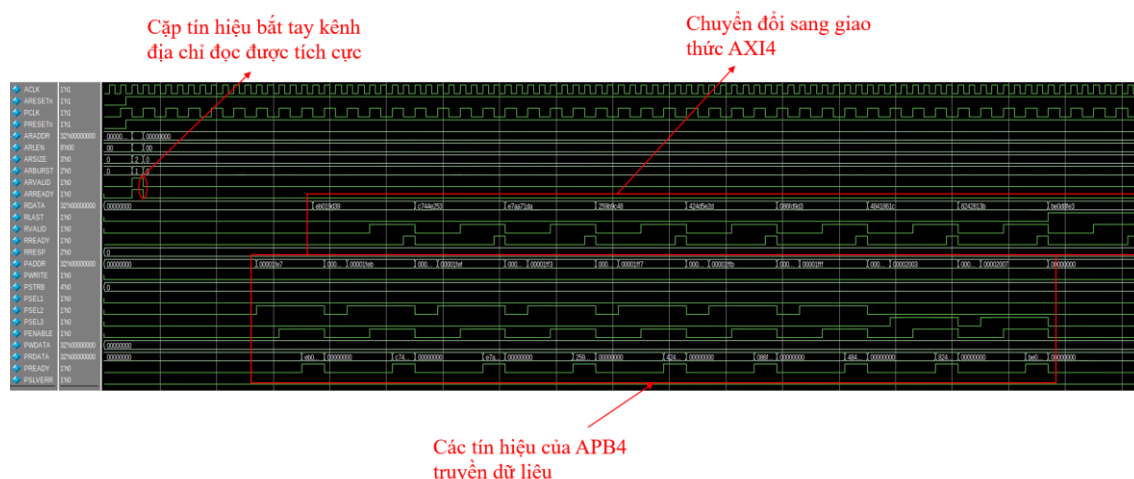
4.2.2.2 Tập kiểm tra 6

Trong tập kiểm tra 6, chúng tôi tiến hành kiểm tra quá trình đọc dữ liệu theo cơ chế INCR. Bắt đầu quá trình đọc theo cơ chế INCR, truyền 8 beat và 4 bytes cho một giao dịch đọc. Các thông số đầu vào được ghi ở bảng 4.7 dưới đây.

Bảng 4.7. Thông số đầu vào của tập kiểm tra 6

TÍN HIỆU ĐẦU VÀO	GIÁ TRỊ
ARADDR	Ngẫu nhiên từ 32'h1FE7 đến 32'h1FF0
ARLEN	8'd7
ARSIZE	3'b00
ARBURST	2'b01

Ta quan sát kết quả mô phỏng trong hình 4.6. dưới đây.



Hình 4.6. Dạng sóng tập kiểm tra 6 của quá trình đọc

Tại thời điểm 25ns, tín hiệu ARVALID và ARREADY cùng tích cực, lúc này kênh địa chỉ đọc hoàn tất việc truyền dữ liệu. Tại 135ns, các tín hiệu của giao thức APB bắt đầu đọc dữ liệu từ ngoại vi. Thời điểm 185ns, bộ chuyển đổi thực hiện việc chuyển đổi beat đầu tiên từ APB4 sang AXI4. Tại thời điểm 835ns, thời điểm ngoại vi bên APB4 hoàn thành việc truyền dữ liệu. Đồng thời, tín hiệu RLAST tích cực, biểu thị transfer cuối cùng trong giao dịch đã được nhận. Tại 915ns, kênh dữ liệu đọc hoàn tất việc đọc dữ liệu. Toàn bộ quá trình chuyển đổi giao thức tuân thủ đúng đặc tả AMBA và không phát sinh lỗi.

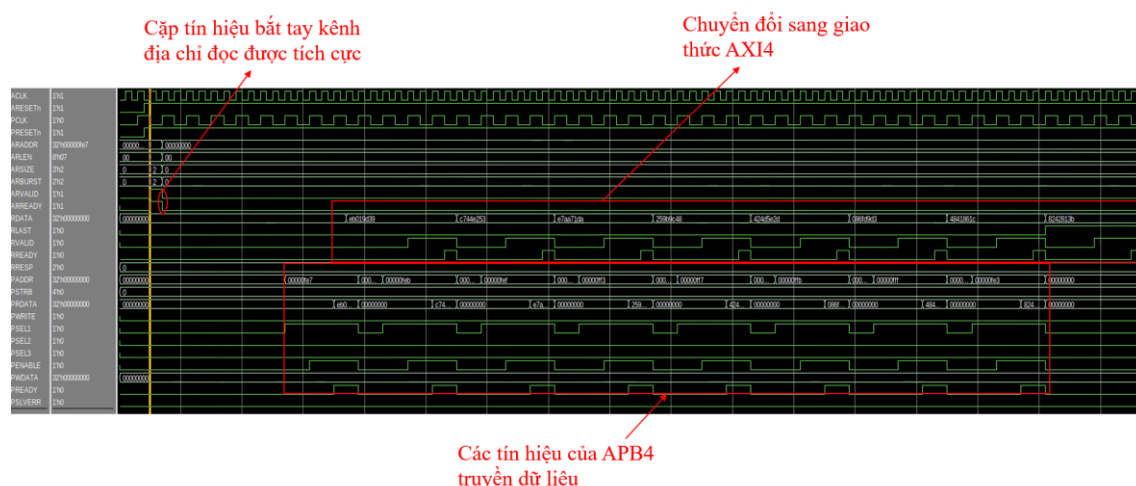
4.2.2.3 Tập kiểm tra 7

Trong tập kiểm tra 7, chúng tôi tiến hành kiểm tra quá trình ghi dữ liệu theo cơ chế WRAP. Bắt đầu quá trình ghi theo cơ chế WRAP, truyền 8 beat và 4 bytes cho một giao dịch đọc. Các thông số đầu vào được ghi ở bảng 4.7. dưới đây.

Bảng 4.8. Thông số đầu vào của tập kiểm tra 7

TÍN HIỆU ĐẦU VÀO	GIÁ TRỊ
ARADDR	Ngẫu nhiên từ 32'hFE7 đến 32'hFF0
ARLEN	8'd7
ARSIZE	3'b10
ARBURST	2'b10

Ta quan sát kết quả mô phỏng trong hình 4.7. dưới đây.



Hình 4.7. Dạng sóng tập kiểm tra 7 của quá trình đọc

Tại thời điểm 25ns, tín hiệu ARVALID và ARREADY cùng tích cực, lúc này kênh địa chỉ đọc hoàn tất việc truyền dữ liệu địa chỉ. Tại 135ns, các tín hiệu của giao thức APB bắt đầu đọc dữ liệu từ ngoại vi. Tại 185ns, các tín hiệu thuộc kênh địa chỉ đọc của giao thức AXI4 bắt đầu truyền dữ liệu ra bên ngoài. Tại 825ns, tín hiệu RLAST tích cực, biểu thị transfer cuối cùng trong giao dịch đã được nhận. Tại 835ns, kênh dữ liệu đọc hoàn tất việc đọc dữ liệu. Toàn bộ quá trình chuyển đổi giao thức tuân thủ đúng đặc tả AMBA và không phát sinh lỗi.

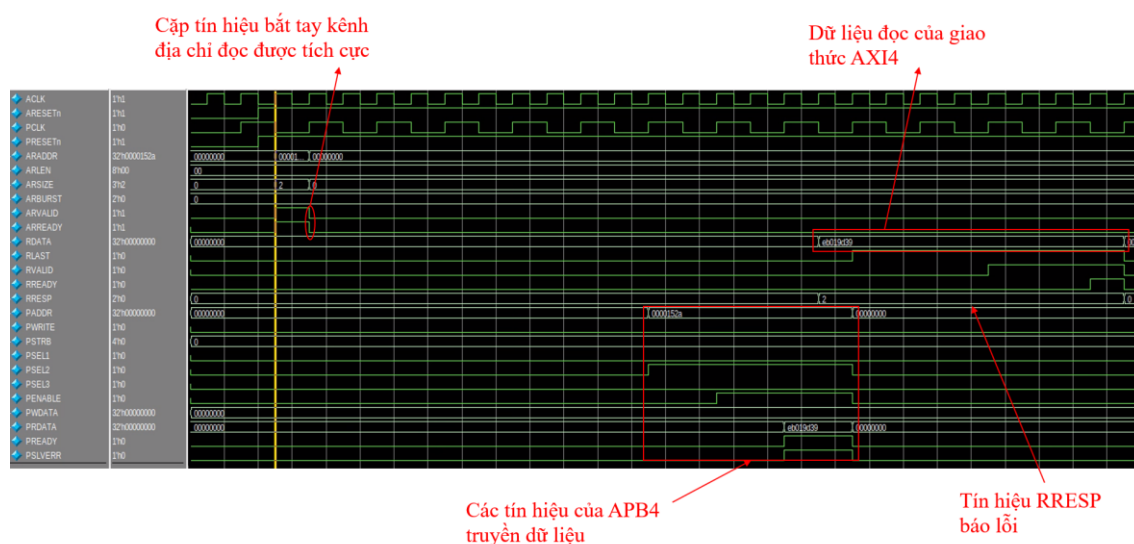
4.2.2.4 Tập kiểm tra 8

Trong tập kiểm tra 8, chúng tôi tiến hành kiểm tra quá trình ghi dữ liệu và trong quá trình đọc dữ liệu, giao thức APB phản hồi có lỗi phát sinh. Các thông số đầu vào được ghi ở bảng 4.8 dưới đây.

Bảng 4.8. Thông số đầu vào của tập kiểm tra 8

TÍN HIỆU ĐẦU VÀO	GIÁ TRỊ
ARADDR	Ngẫu nhiên từ 32'h1000 đến 32'h2FFF
ARLEN	8'd0
ARSIZE	3'b00
ARBURST	2'b00
PSLVERR	1

Ta quan sát kết quả mô phỏng trong hình 4.8. dưới đây.



Hình 4.8. Dạng sóng tập kiểm tra 8 của quá trình đọc

Tại thời điểm 25ns, tín hiệu ARVALID và ARREADY cùng tích cực, lúc này kênh địa chỉ đọc hoàn tất việc truyền dữ liệu địa chỉ. Tại 135ns, các tín hiệu của giao thức APB4 bắt đầu đọc dữ liệu từ ngoại vi. Tại 185ns, các tín hiệu thuộc kênh địa chỉ đọc của giao thức AXI4 bắt đầu truyền dữ liệu ra bên ngoài. Đồng thời tín hiệu RLAST tích cực, biểu thị transfer cuối cùng trong giao dịch đã được truyền đi. Tại 275ns, kênh dữ liệu đọc hoàn tất việc đọc dữ liệu. Toàn bộ quá trình chuyển đổi giao thức tuân thủ đúng đặc tả AMBA và không phát sinh lỗi.

4.2.3 Hoạt động đọc và ghi dữ liệu (Testcase 9)

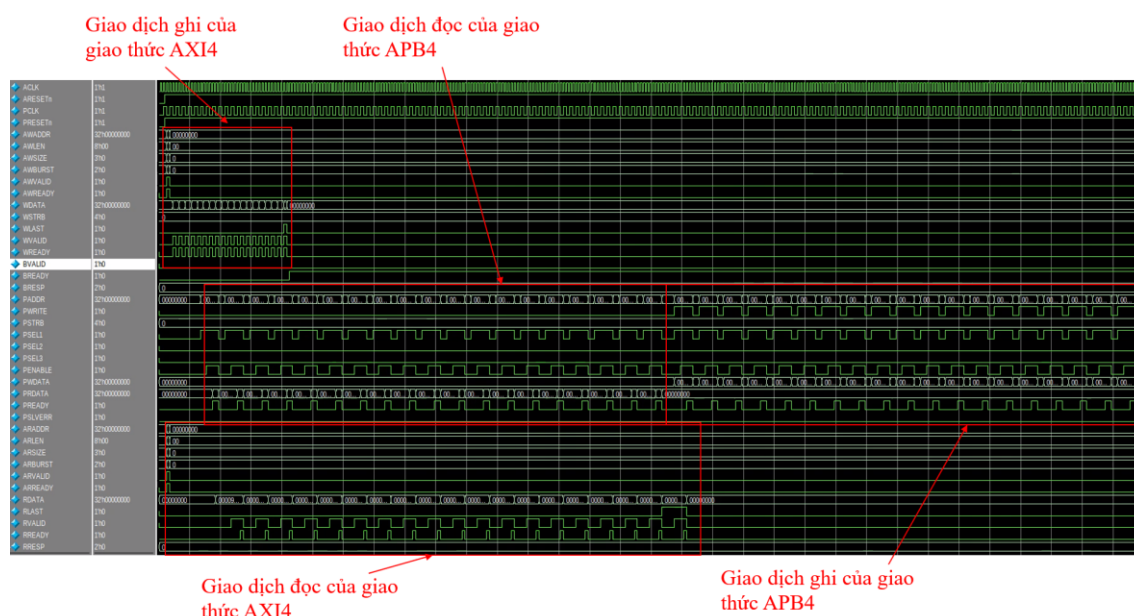
Xây dựng các tệp kiểm tra của quá trình đọc với mục tiêu kiểm thử các trường hợp để mô phỏng quá trình ghi và đọc cùng lúc khi có yêu cầu từ AXI.

Trong tệp kiểm tra 9, chúng tôi tiến hành kiểm tra quá trình đọc và ghi dữ liệu dữ liệu cùng một lúc. Các thông số đầu vào được ghi ở bảng 4.9 dưới đây.

Bảng 4.9. Thông số đầu vào của tệp kiểm tra 9

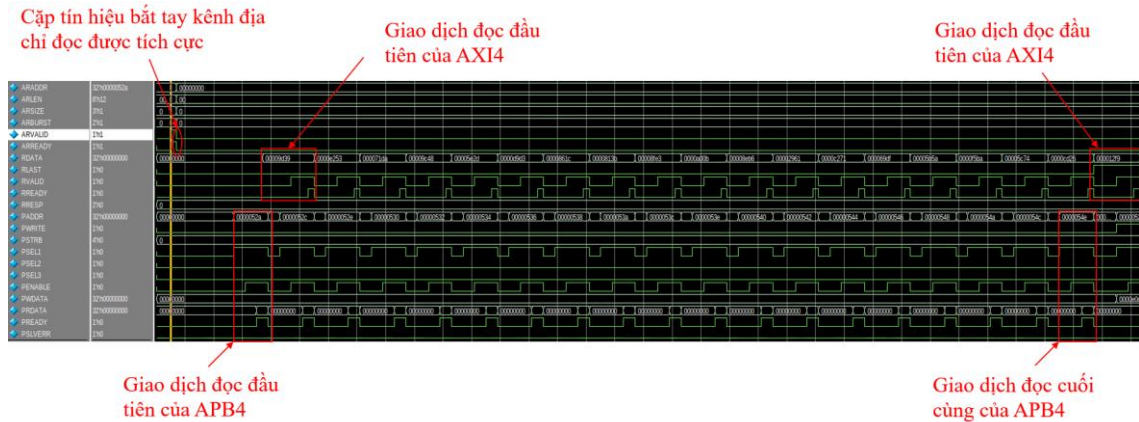
TÍN HIỆU ĐẦU VÀO	GIÁ TRỊ
AWADDR	Ngẫu nhiên từ 32'h00 đến 32'h2FFF
AWLEN	\$random()
AWSIZE	\$random()
AWBURST	\$random()
ARADDR	Ngẫu nhiên từ 32'h00 đến 32'h2FFF
ARLEN	\$random()
ARSIZE	\$random()
ARBURST	\$random()

Ta quan sát kết quả mô phỏng trong hình 4.9. dưới đây.



Hình 4.9. Dạng sóng tệp kiểm tra 9 của quá trình đọc và ghi

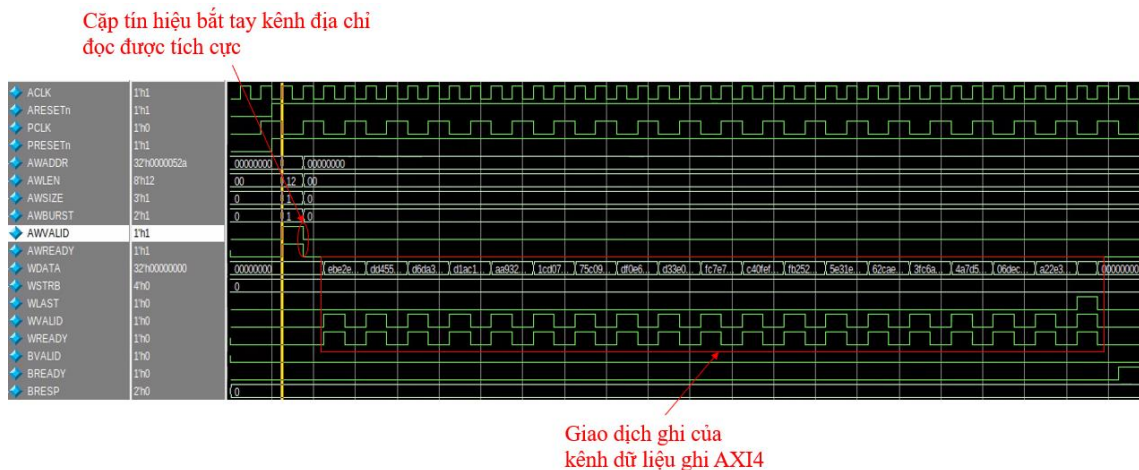
Để nhìn rõ được sóng tín hiệu, chúng tôi tiến hành chia ra phân tích giao dịch đọc và giao dịch ghi riêng. Vì giao dịch đọc được chuyển đổi đầu tiên nên chúng tôi tiến hành phân tích trước.



Hình 4.10. Dạng sóng tập kiểm tra 9 của quá trình đọc

Tại thời điểm 25ns, tín hiệu ARVALID tích cực. Tại thời điểm 45ns, ARVALID và ARREADY cùng tích cực, lúc này kênh địa chỉ đọc hoàn tất việc truyền dữ liệu. Tại 135ns, bắt đầu đọc dữ liệu từ ngoại vi. Tại 185ns, bắt đầu truyền dữ liệu của kênh đọc dữ liệu. Tại 1635ns, giao dịch đọc cuối cùng từ ngoại vi được truyền đi thành công. Tại 1715ns, hoàn tất quá trình chuyển đổi giao dịch đọc. Quá trình chuyển đổi thành công và không phát sinh lỗi.

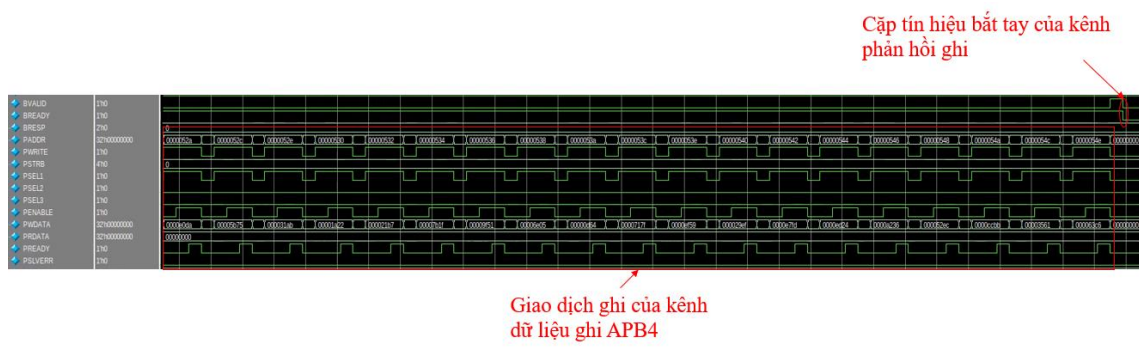
Tiếp theo, tiến hành phân tích giao dịch ghi, đầu tiên là kênh dữ liệu ghi.



Hình 4.11. Dạng sóng tập kiểm tra 9 của giao dịch ghi AXI4

Tại 25ns, tín hiệu AWVALID tích cực. Tại 35ns, AWVALID và AWREADY cùng tích cực, lúc này kênh địa chỉ ghi hoàn tất truyền dữ liệu. Tại 45ns, kênh dữ liệu ghi bắt đầu ghi truyền dữ liệu. Đến thời điểm 415ns, dữ liệu

ghi cuối cùng được truyền đi. Quá trình truyền dữ liệu ghi hoàn tất. Tại 425ns, tín hiệu BREADY tích cực, chờ đến khi bên phía APB4 hoàn thành việc truyền dữ liệu ghi và trả về phản hồi BRESP.



Hình 4.12. Dạng sóng tập kiểm tra 9 của giao dịch ghi APB4

Tại 1673ns, các tín hiệu APB4 bắt đầu chuyển đổi giao dịch ghi. Đến thời điểm 3175ns, dữ liệu ghi cuối cùng được truyền đi. Đồng thời, tại thời điểm đó, cặp tín hiệu BVALID và BREADY phản hồi lại cho giao thức AXI4. Quá trình ghi hoàn tất và không phát sinh lỗi.

4.3 KẾT QUẢ BÁO CÁO ĐỘ BAO PHỦ VÀ ĐỘ BAO PHỦ CHỨC NĂNG

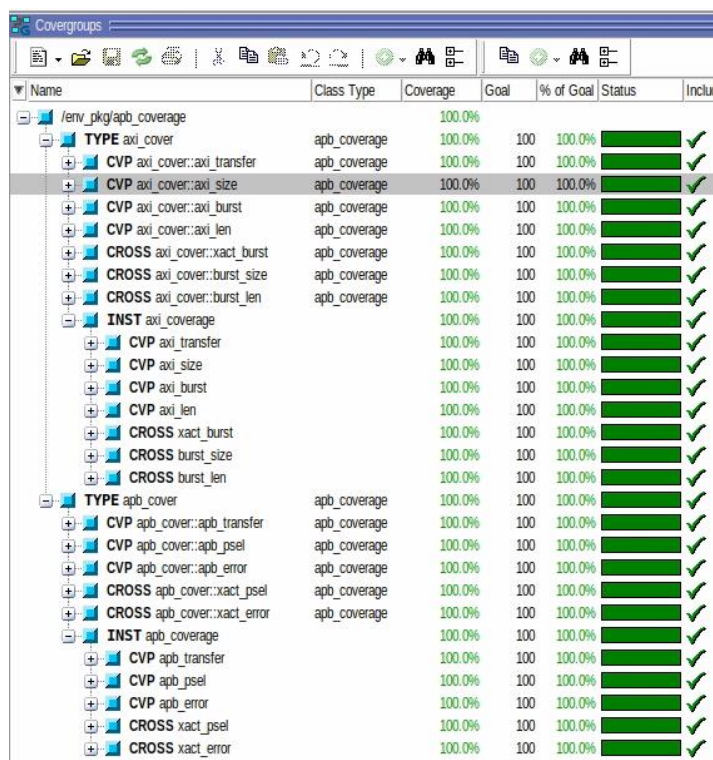
Hình 4.13. dưới đây là kết quả báo cáo độ bao phủ của thiết kế, cho thấy các tập kiểm tra đã bao phủ được 81.8% các logic và nhánh chức năng của thiết kế. Việc độ bao phủ không đạt 100% là do một số trường hợp hiếm gặp, các nhánh hoặc các điều kiện không được kích hoạt trong quá trình mô phỏng. Tuy nhiên, kết quả này vẫn thể hiện được mức độ đầy đủ và độ tin cậy của quá trình kiểm chứng thiết kế trong phạm vi đề tài.

Instance	Design unit	Design unit (Top Category)	Visibility	Cover Options	Total coverage	Stmts hit	Stmts missed	Stmt %	Stmt graph	Branch count	Branches hit	Branches missed	Branch %	Branch graph	Toggle nodes	Toggles hit	Toggles missed	Toggle %	Toggle graph
testbench	testbench	Module	DU Instance	+acc=<n...	+cover=boefst	82.2%	346	9	97.5%	180	169	11	93.9%		3928	3384	544	86.2%	
axi_if	axi_if	Interface	DU Instance	+acc=<n...	+cover=boefst	97.2%	10	0	100%						352	332	20	94.3%	
apb_if	apb_if	Interface	DU Instance	+acc=<n...	+cover=boefst	100.0%	5	0	100%						218	218	0	100%	
axi	AXI to AP...	Module	DU Instance	+acc=<n...	+cover=boefst	81.8%	309	9	97.2%	180	169	11	93.9%		3350	2826	524	84.4%	
axi_slave	axi_slave	Module	DU Instance	+acc=<n...	+cover=boefst	77.9%	201	0	100%	94	92	2	97.9%		1760	1604	156	91.1%	
apb_master	apb_master	Module	DU Instance	+acc=<n...	+cover=boefst	83.1%	108	9	92.3%	86	77	9	89.5%		1742	1366	376	78.4%	
uvm_pkg	uvm_pkg	VPackage	Package	+acc=<n...	+cover=boefst	16.7%	58	163	26.2%	223	16	207	7.17%						
apb_pkg	apb_pkg	VPackage	Package	+acc=<n...	+cover=boefst	25.0%	217	266	44.9%	380	75	305	19.7%						
axi_pkg	axi_pkg	VPackage	Package	+acc=<n...	+cover=boefst	62.3%	95	36	72.5%	143	62	81	43.4%						
env_pkg	env_pkg	VPackage	Package	+acc=<n...	+cover=boefst	35.8%	834	641	56.5%	1018	154	864	15.1%						
seq_pkg	seq_pkg	VPackage	Package	+acc=<n...	+cover=boefst	59.0%	982	200	83.1%	20	7	13	35%						
test_pkg	test_pkg	VPackage	Package	+acc=<n...	+cover=boefst														

Hình 4.13. Kết quả báo cáo độ bao phủ của bộ chuyển đổi

Kết quả báo cáo độ bao phủ chức năng được thể hiện ở hình 4.14. dưới đây, được xây dựng để kiểm chứng các chức năng chính của giao thức AXI4 như loại

giao dịch, kích thước dữ liệu, cơ chế truyền, độ dài dữ liệu. Đồng thời cũng kiểm chứng các chức năng của giao thức APB4 như loại giao dịch, tín hiệu chọn Slave và trạng thái lỗi. Kết quả đạt được cho thấy các kịch bản kiểm thử chức năng quan trọng của thiết kế đã được xây dựng và kiểm tra đầy đủ trong môi trường mô phỏng.



Hình 4.14. Kết quả báo cáo độ bao phủ chức năng của bộ chuyển đổi

4.4 ĐÁNH GIÁ TÀI NGUYÊN SỬ DỤNG

Các thông số tài nguyên sử dụng cho bộ chuyển đổi AXI4 sang APB4 được tổng hợp qua bảng 4.10. bên dưới, thiết lập của bộ FPGA Board Spartan-7 SP701 Evaluation Platform (xc7s100fgga676-2).

Bảng 4.10. Thông số tài nguyên hệ thống

TÀI NGUYÊN	SỬ DỤNG	CÓ SẴN	TỈ LỆ (%)
LUT	2084	64000	3.26
FF	5050	128000	3.95
IO	284	400	71.00
BUFG	2	32	6.25

Kết quả tổng hợp trên cho thấy hệ thống được thiết kế đã sử dụng 2084 khối chức năng rời rạc có thể lập trình được (Look Up Table – LUT) chỉ chiếm 3.26% tài nguyên trong xc7s100fpga676-2 và phần lớn sử dụng cho các phép tính logic. Số lượng Flip Flip đã sử dụng 5050 slice, chỉ chiếm 3.95% tài nguyên, số lượng chân được sử dụng là 284 đơn vị, chiếm 71.00%, và 2 clock buffer (BUFG) chiếm 6.25%.

4.5 ĐÁNH GIÁ TẦN SỐ VÀ CÔNG SUẤT CỦA THIẾT KẾ

Tần số và công suất của thiết kế được ước lượng và tính toán trên kit Spartan-7 SP701 Evaluation Platform thông qua phần mềm Vivado được thể hiện ở bảng 4.11.

Bảng 4.11. Công suất của thiết kế với các tần số khác nhau

TẦN SỐ	ACLK: 25MHz	ACLK: 125MHz	ACLK: 200MHz
	PCLK: 20MHz	PCLK: 50MHz	PCLK: 100MHz
Total On-Chip Power (W)	0.105	0.132	0.156
Dynamic Power (W)	0.008	0.035	0.060
Static Power (W)	0.097	0.097	0.097
Thermal Margin (oC)	59.7	59.7	59.6
Junction Temperature (oC)	25.3	25.32	25.4

Dựa vào kết quả tính được trên bảng 4.11. ta thấy tần số hoạt động tối đa của hệ thống là 200MHz với xung nhịp cho giao thức AXI và 100 MHz với xung nhịp cho giao thức APB. Nhìn chung, công suất của hệ thống thuộc về công suất thấp. Hệ thống hoạt động tốt trong điều kiện nhiệt độ phòng bình thường.

Về mặt timing, ta chọn tần số tối đa của ACLK và PCLK là 200MHz và 100MHz, báo timing trong bảng 4.12 dưới đây.

Bảng 4.12. Báo cáo về mặt timing của hệ thống

Worst Negative Slack - WNS (ns)	0.230
TNS Total Endpoints	7073
Worst Hold Slack – WHS (ns)	0.117

THS Total Endpoints	7073
Worst Pulse Width Slack – WPWS (ns)	2.000
TPWS Total Endpoints	5052

Ta có thể tính được giá trị tần số tối đa của hệ thống theo công thức $F_{max} = 1/(T-WNS)$. Trong đó F_{max} là tần số tối đa thiết kế có thể chạy trên phần cứng trong khi thực thi, T là chu kỳ xung và WNS là độ trễ âm tẹ nhất của đường tín hiệu xung nhịp. Vậy tần số tối đa của xung nhịp cho giao thức AXI4 và APB4 lần lượt là:

$$F_{max, ACLK} = \frac{1}{5 - 0.230} = 209.643MHz$$

$$F_{max, PCLK} = \frac{1}{10 - 0.230} = 102.354MHz$$

CHƯƠNG 5

KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

5.1 KẾT LUẬN

Sau quá trình thực hiện thiết kế và kiểm thử bộ chuyển đổi AXI4 sang APB4 sử dụng UVM, đề tài có một số ưu điểm như sau:

- Bộ chuyển đổi được thiết kế và đánh giá trên Kit FPGA “xc7s100fgga676-2”, hoạt động ổn định ở tần số 209MHz đối với giao thức AXI4 và 102MHz đối với giao thức APB4.
- Bộ chuyển đổi có sử dụng tham số để cho phép người dùng cấu hình linh động địa chỉ cho các khối APB4 con.
- Các tệp kiểm tra được mô phỏng ở tần số 200MHz cho AXI4, 100MHz cho APB4, bao phủ nhiều trường hợp kiểm thử khác nhau, bao gồm lựa chọn ba khối APB khác nhau, kích thước và độ dài dữ liệu khác nhau, cũng như kiểm tra lỗi khi bên APB4 báo lỗi.
- Thiết kế đạt 100% độ bao phủ chức năng, đảm bảo các chứng năng chính của bộ chuyển đổi như đọc dữ liệu, ghi dữ liệu, các loại truyền dữ liệu, độ dài dữ liệu khác nhau.
- Đánh giá về mặt tần số, thời gian, cũng như hiệu năng của bộ chuyển đổi.

Bên cạnh đó, đề tài còn một số nhược điểm:

- Thiết kế chỉ chuyển được cho ba khối APB con, do đó chưa đáp ứng được các hệ thống có quy mô rộng lớn hơn.
- Việc chuyển đổi dữ liệu sau khi hoàn thành nhận dữ liệu đầu tiên từ AXI4 mất khoảng 7 đến 9 chu kỳ xung ACLK mới có thể bắt đầu chuyển đổi dữ liệu sang APB4.

- Môi trường thiết kế chưa bao quát được toàn bộ thiết kế, độ bao phủ chỉ đạt 81.8% và các tệp kiểm tra chưa kiểm chứng thiết kế ở nhiều tần số khác nhau cho hai giao thức AXI4 và APB4.
- Đề tài chưa thực nghiệm trên phần cứng thực tế, mà mới dừng lại ở mức mô phỏng và phân tích sau tổng hợp.

5.2 HƯỚNG PHÁT TRIỂN

Dựa trên kết quả đạt được và các hạn chế của đề tài, người thực hiện đề xuất một số hướng phát triển trong tương lai cho đề này như sau:

- Mở rộng môi trường kiểm thử UVM nhằm nâng cao độ bao phủ thiết kế.
- Triển khai cơ chế hỗ trợ chuyển đổi cho số lượng lớn khối APB con, cho phép cấu hình linh hoạt địa chỉ chuyển đổi thông qua tham số, phù hợp với các hệ thống SoC có độ phức tạp cao.
- Mở rộng nghiên cứu sang các bộ chuyển đổi giao thức khác như AXI4-Lite, I2C, UART và nhiều giao thức khác, nhằm tăng tính ứng dụng của đề tài trong các hệ thống nhúng hiện đại.

TÀI LIỆU THAM KHẢO

- [1] A. Ramesh Babu, P. Anand, Y. Kim, and S. Jadhav, "System Verilog versus UVM-based Verification of AXI4-Lite Arbitration," *Proc. IEEE SoutheastCon*, p. 350–357, Apr. 2023.
- [2] S. A. Pharate, M. P. Kadam, A. D. Sawant, R. K. Shastri, and A. Gangad, "Verification of AMBA-APB Protocol using UVM," *IJIREEICE*, vol. 12, p. 1, 2024.
- [3] Đ. V. Trung, "Thiết kế mô hình RTL và kiểm thử UVM cho giao tiếp APB-AHB Bridge," luận văn, Học viện Công nghệ Bưu chính Viễn thông, 2025.
- [4] Vinh T. Q. and Hung H., "UVM-Based Verification of AMBA AHB-to-APB Bridge Using Modularized Interfaces and Reactive Slaves," *J. Phys.: Conf. Ser*, vol. 3020, p. 1, 2025.
- [5] Synopsys, "What is a System on Chip (SoC)?," Synopsys Blog, [Online]. Available: <https://www.synopsys.com/blogs/chip-design/system-on-chip.html>. [Accessed 26 Dec 2025].
- [6] N. Quan, "Background: SoC (System on Chip)," Nguyen Quan ICD, Aug 2018. [Online]. Available: <https://nguyenquanica.blogspot.com/2018/05/> [Accessed 26 Dec 2025].
- [7] Ansys, "What is a system on a chip (SoC)?," Ansys Blog, 31 Oct 2023. [Online]. Available: <https://www.ansys.com/blog/what-is-system-on-a-chip>. [Accessed 26 Dec 2025].
- [8] N. Quan, "Bus bài 1: Giao thức AMBA AXI," Nguyen Quan ICD, Aug 2018. [Online]. Available: <https://nguyenquanica.blogspot.com/>. [Accessed 30 Dec 2025].
- [9] ARM Limited, "AMBA® AXI Protocol Specification," *ARM IHI 0022E*, Feb 2013.
- [10] ARM Limited, "AMBA® APB Protocol Specification," *ARM IHI 0024E*, 2023.

- [11] Accellera Systems Initiative, "Universal Verification Methodology (UVM) 1.2 User's Guide," *Accellera*, 8 Oct 2015.
- [12] Mentor Graphics, "Universal Verification Methodology (UVM) Cookbook," *Wilsonville, OR, USA: Mentor Graphics Corp.*, 2013.
- [13] Thang P. V., Quang N. H., Tam N. H., Tien P. V., and Nguyen T. N. H., "First X2P bridge," 2026. [Online]. Available: <https://github.com/nguyenquanicd/FirstX2P>. [Accessed Jan 2026].
- [14] minhcly95, "AXI-APB Bridge," 2024. [Online]. Available: https://github.com/minhcly95/ChipDesignPractice/tree/main/axi_apb_bridge. [Accessed Jan 2026].

PHỤ LỤC A

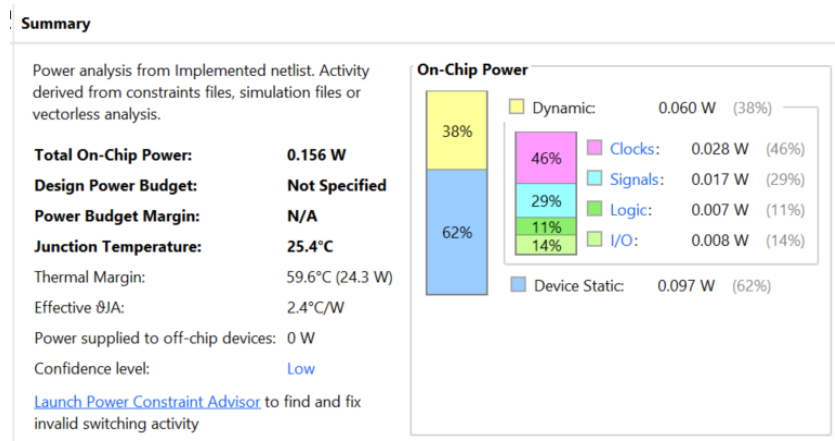
MINH CHỨNG KẾT QUẢ

Hình phụ lục 1 cho thấy tài nguyên sử dụng của thiết kế sử dụng công cụ Vivado để tính toán.

Utilization			
		Post-Synthesis	Post-Implementation
Graph Table			
Resource	Utilization	Available	Utilization %
LUT	2084	64000	3.26
FF	5050	128000	3.95
IO	284	400	71.00
BUFG	2	32	6.25

Hình phụ lục 1. Minh chứng tài nguyên sử dụng của thiết kế

Hình phụ lục 2 minh chứng cho kết quả về công suất tiêu thụ của thiết kế ở tần số 200MHz đối với giao thức AXI4 và 100MHz đối với giao thức PCLK.



Hình phụ lục 2. Minh chứng công suất tiêu thụ ở tần số 200MHz, 100MHz

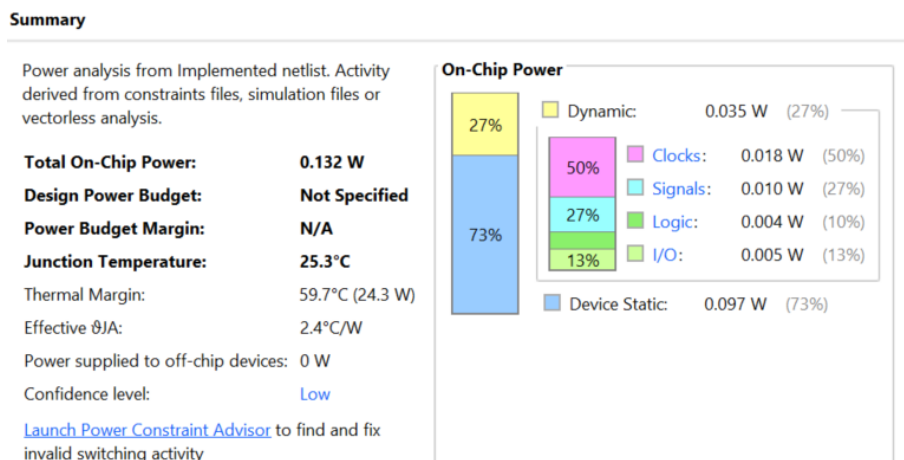
Hình phụ lục 3 minh chứng kết quả định thời của thiết kế ở tần số 200MHz đối với giao thức AXI4 và 100MHz đối với giao thức PCLK.

Setup	Hold	Pulse Width
Worst Negative Slack (WNS): 0.230 ns	Worst Hold Slack (WHS): 0.117 ns	Worst Pulse Width Slack (WPWS): 2.000 ns
Total Negative Slack (TNS): 0.000 ns	Total Hold Slack (THS): 0.000 ns	Total Pulse Width Negative Slack (TPWS): 0.000 ns
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
Total Number of Endpoints: 7073	Total Number of Endpoints: 7073	Total Number of Endpoints: 5052

All user specified timing constraints are met.

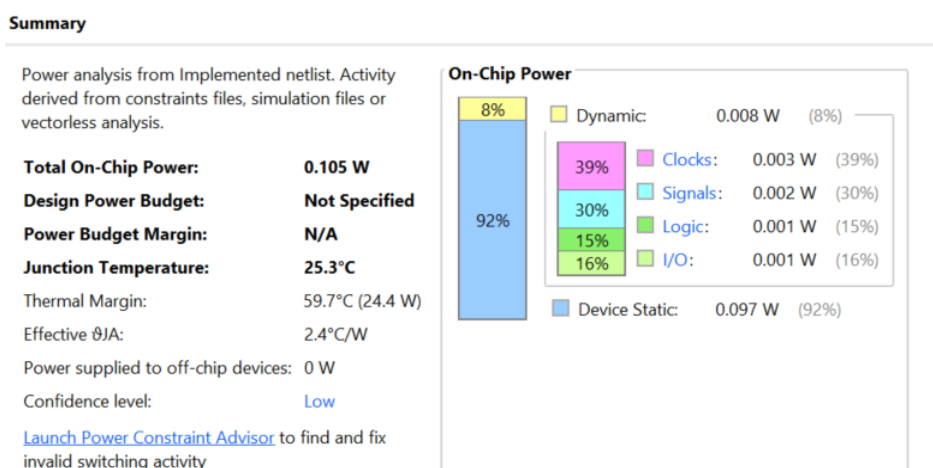
Hình phụ lục 3. Minh chứng định thời ở tần số 200MHz, 100MHz

Hình phụ lục 4 minh chứng cho kết quả về công suất của thiết kế ở tần số 125MHz đối với giao thức AXI4 và 50MHz đối với giao thức PCLK.



Hình phụ lục 4. Minh chứng công suất tiêu thụ ở tần số 125MHz, 50MHz

Hình phụ lục 5 minh chứng cho kết quả về công suất của thiết kế ở tần số 25MHz đối với giao thức AXI4 và 20MHz đối với giao thức PCLK.



Hình phụ lục 5. Minh chứng công suất tiêu thụ ở tần số 25MHz, 20MHz

PHỤ LỤC B

ĐƯỜNG DẪN LƯU MÃ NGUỒN

Các mã nguồn và các tài liệu liên quan được đặt trong đường dẫn:

https://github.com/ngdkvy/DA2_Design-and-Verify-AXI4-to-APB4-bridge-using-UVM